

CPU-bound Monte Carlo 시뮬레이션 기반 운영체제 병렬 처리 구조 비교 및 성능 분석

송실대학교 전자정보공학부 IT 융합

운영체제(이찬호 교수)

김태환 성도연 유지원 정재훈 정종근

0. 초록

본 프로젝트는 Monte Carlo 기반 차량 추종 위험 계산을 CPU-bound workload로 사용하여 운영체제의 병렬 처리 구조를 비교·분석하는 것을 목표로 한다. 동일한 workload를 Sequential, Thread, Process, Hybrid, Pipeline 구조로 실행하고, 각 구조의 정확성, 실행 시간, speedup, efficiency, CPU utilization, overhead를 측정하였다. 본 프로젝트에서 차량 추종 위험 계산은 실제 차량 시뮬레이터 구현이 아니라, 운영체제 실행 구조의 성능 차이를 분석하기 위한 실험용 workload로 사용된다.

초기에는 단순 병렬 처리 구조를 구현하는 데 초점을 두었으나, 프로젝트가 진행되면서 race condition, lock contention, process 간 결과 공유, IPC 비용, queue synchronization, worker 수 증가에 따른 efficiency 감소 문제가 확인되었다. 이를 해결하기 위해 thread 구조에서는 nosync, mutex, reduce 방식을 비교하였고, process 구조에서는 fork(), waitpid(), pipe IPC, shared memory IPC를 구현하였다. 또한 process와 thread를 결합한 hybrid 구조와 TaskQueue, MergeQueue, condition variable 기반 pipeline 구조를 추가하여 운영체제 병렬 처리 개념을 폭넓게 비교할 수 있도록 확장하였다.

실험은 실제 Monte Carlo workload를 기준으로 수행하였다. 주요 실험 조건은 120,000,000 trials, steps=50, 각 case 5회 반복이며, 모든 정상 mode는 valid=1, hist_sum=trials, checksum 일치를 만족해야 성능 비교 대상으로 사용하였다. 최종 결과에서 4-worker 주요 구조는 약 92.5~94.0% 수준의 efficiency를 보였고, worker당 CPU utilization은 약 99.2~99.8% 수준으로 측정되었다. 이는 실제 workload에서도 4개 worker까지는 N-core 활용에 상당히 근접했음을 의미한다.

반면 8-worker 조건에서는 CPU utilization은 높게 유지되었지만 efficiency는 감소하였다. 이는 CPU를 바쁘게 사용하는 것과 실행 시간이 이상적으로 감소하는 것이 동일하지 않음을 보여준다. 병렬 처리 성능은 CPU 사용률뿐 아니라 synchronization overhead, IPC overhead, queue overhead, merge overhead, scheduling 영향까지 함께 고려해야 한다. 따라서 본 프로젝트의 결론은 특정 실행 구조가 항상 최적이라는 것이 아니라, 동일한 CPU-bound workload라도 운영체제 실행 구조에 따라 정확성, overhead, scalability가 달라진다는 점이다.

1. 시스템 개요

1-1. 프로젝트 개요

본 프로젝트는 Monte Carlo 기반 차량 추종 위험 계산을 CPU-bound workload 로 사용하여, 운영체제의 병렬 처리 구조를 비교·분석하는 것을 목표로 한다. 동일한 workload 를 Sequential, Thread, Process, Hybrid, Pipeline 구조로 실행하고, 각 구조의 정확성, 실행 시간, speedup, efficiency, CPU utilization, overhead 를 측정한다.

이 프로젝트의 목적은 실제 차량 시뮬레이터를 정밀하게 구현하는 것이 아니다. 차량 추종 위험 계산은 운영체제 실행 구조를 비교하기 위한 실험용 workload 이다. 핵심 분석 대상은 child process, POSIX thread, synchronization, IPC, queue, merge 구조가 병렬 처리 성능에 어떤 영향을 주는지이다.

최종 시스템은 단순히 가장 빠른 구조를 찾는 것이 아니라, 각 실행 구조가 어떤 조건에서 유리하고 어떤 overhead 를 가지는지 분석하는 데 초점을 둔다. 따라서 모든 성능 비교는 먼저 valid, hist_sum, checksum 으로 결과 정확성을 확인한 뒤 수행한다.

1-2. 주제 선정 배경

초기 아이디어는 이미지 필터 병렬 처리였다. 이미지 필터는 이미지를 영역별로 나누어 process 나 thread 가 처리할 수 있으므로 병렬화가 가능하다. 그러나 이미지 처리에서는 파일 I/O, 메모리 접근 패턴, cache locality, memory bandwidth 가 실행 시간에 크게 영향을 줄 수 있다. 이 경우 실행 시간이 줄어들더라도 그 원인이 CPU 병렬화 때문인지, 메모리 접근이나 I/O 때문인지 명확히 분리하기 어렵다.

이에 따라 최종 workload 를 Monte Carlo 기반 차량 추종 위험 계산으로 변경하였다. Monte Carlo 방식은 각 trial 이 독립적으로 수행되고, 반복 계산 중심으로 구성할 수 있다. 따라서 worker 수를 늘렸을 때 CPU-bound 병렬 처리 효과를 비교하기에 적합하다.

본 프로젝트에서 각 trial 은 deterministic seed 를 기반으로 차량 상태를 생성하고, 정해진 time step 동안 상대 거리와 위험도를 계산한다. 결과는 histogram 과 checksum 에 반영된다. 이 구조는 thread, process, hybrid, pipeline 구조로 쉽게 분할할 수 있고, 실행 구조가 달라도 같은 trial index 는 같은 결과를 생성하도록 설계할 수 있다.

1-3. 프로젝트 목표

본 프로젝트의 목표는 다음과 같다.

1. 실제 Monte Carlo workload 를 이용해 CPU-bound 병렬 처리 구조를 구현한다.
2. pthread 기반 thread 병렬화를 구현하고 synchronization 문제를 확인한다.
3. fork() 기반 child process 구조를 구현하고 IPC 를 통해 결과를 전달한다.
4. pipe 와 shared memory 방식의 IPC 구조를 비교한다.
5. process 와 thread 를 결합한 hybrid 구조를 구현한다.
6. TaskQueue 와 MergeQueue 기반 pipeline 구조를 구현한다.
7. valid, hist_sum, checksum 으로 모든 정상 mode 의 정확성을 검증한다.

8. 1, 2, 4, 8 worker 조건에서 speedup, efficiency, CPU utilization 을 측정한다.
9. CPU utilization 과 scaling efficiency 가 서로 다른 지표임을 분석한다.

1-4. 프로젝트 변화 과정

프로젝트는 초기 아이디어에서 최종 실험 시스템까지 단계적으로 확장되었다. 핵심 변화는 “단순 병렬 처리 구현”에서 “운영체제 실행 구조별 trade-off 분석”으로 발전한 점이다.

표 1. 프로젝트 변화 과정

단계	구현/설계 상태	문제 인식	개선 방향
초기 계획	이미지 필터 병렬 처리	I/O, 메모리 접근, cache 영향으로 CPU 병렬화 효과 분리 어려움	CPU-bound workload 필요
방향 조정	Monte Carlo 차량 추종 위험 계산	실제 차량 시뮬레이터로 오해될 수 있음	OS 병렬 처리 분석용 workload 로 정의
중간 구현	Sequential, Thread, nosync, mutex, reduce	thread 만으로는 process, IPC 설명 부족	child process 와 IPC 추가
구조 확장	Process, Shared Memory, Hybrid 추가	process/thread 단일 구조 비교 한계	process + pthread 계층 구조 구현
Pipeline 추가	TaskQueue, MergeQueue, Aggregator	final reduce 만으로 producer-consumer 구조 설명 부족	queue 기반 pipeline 구현
최종 검증	실제 Monte Carlo workload scaling 측정	실행 시간만으로 core 활용 판단 어려움	CPU utilization, core 별 utilization 함께 측정

1-5. 최종 시스템 구성 및 동작 흐름

최종 시스템은 CLI 옵션을 입력받아 실행 mode 와 workload 조건을 설정한 뒤, 동일한 Monte Carlo workload 를 선택된 실행 구조로 처리한다. 계산이 끝나면 mode 별 partial result 를 병합하고, valid, hist_sum, checksum 으로 정확성을 검증한다. 이후 실행 시간, speedup, efficiency, CPU utilization, stage time 을 CSV 로 저장한다.

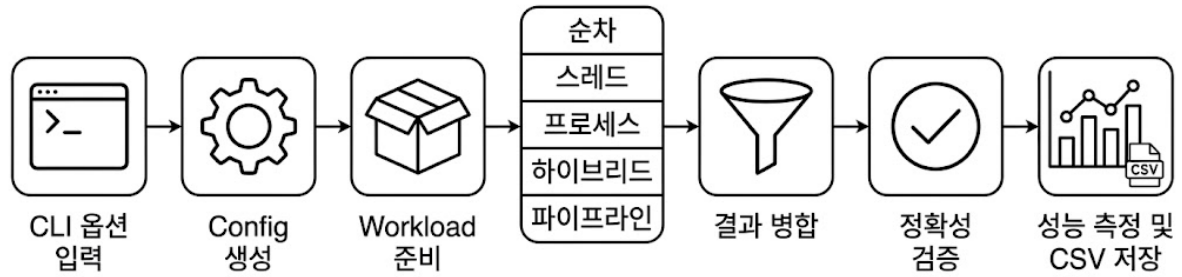


그림 1. 동작 흐름도

1-6. 실행 구조별 역할

최종 시스템의 실행 구조는 모두 같은 workload 를 처리하지만, 사용하는 운영체제 개념과 overhead 요인이 다르다.

표 2. 실행 구조별 역할

실행 구조	핵심 역할	주요 OS 개념	분석 포인트
Sequential	기준 실행 시간 제공	단일 실행 흐름	speedup 계산 기준
Thread	pthread 기반 병렬 처리	thread, shared address space	synchronization, local reduce
Process	child process 기반 병렬 처리	fork, waitpid, address space	IPC, process isolation
Shared Memory IPC	child 결과 전달	mmap, shared memory	copy overhead 감소, slot 설계
Hybrid	process + thread 결합	process group, pthread	계층적 병렬화
Pipeline	queue 기반 작업 분배	producer-consumer, mutex, condition variable	queue overhead, dynamic scheduling

1-7. 병렬처리 가능성 분석

본 프로젝트에서 병렬화의 핵심 대상은 Monte Carlo trial 계산이다. 각 trial 은 독립적으로 수행되므로 thread, process, hybrid 구조로 나누기 쉽다. 반면 histogram update, result merge, IPC, queue operation 은 synchronization 또는 overhead 가 발생할 수 있는 구간이다.

따라서 최종 시스템은 trial 계산을 주요 병렬화 대상으로 두고, synchronization, IPC, merge, queue 는 overhead 분석 대상으로 분리한다. 이 구조를 통해 단순 실행 시간뿐 아니라 성능 저하의 원인을 함께 분석할 수 있다.

표 3. 병렬처리 가능성 및 평가 기준

구성 요소	병렬화 가능성	주요 이슈	적용 구조	평가 기준
Monte Carlo trial 계산	높음	작업 분할 방식	thread, process, hybrid	실행 시간, speedup
TTC/risk 계산	높음	worker 별 계산량	all worker modes	compute time
Histogram update	주의 필요	shared result 충돌	mutex, reduce	valid, checksum
Result merge	제한적	partial result 병합 비용	reduce, pipeline	merge time
IPC	process 에서 필수	주소 공간 분리	pipe, shared memory	IPC overhead
Queue scheduling	조건부 가능	queue/condvar overhead	pipeline	final vs interactive
CPU core 활용	측정 대상	실제 core 사용률	real workload	CPU%, Util/Worker
순차 구간	낮음	speedup 제한	pre/post stage	efficiency

1-8. 최종 평가 지표

최종 평가는 단일 실행 시간이 아니라 네 가지 기준으로 수행한다.

첫째, 정확성이 보장되어야 한다. 둘째, 실행 시간이 얼마나 감소했는지 확인해야 한다. 셋째, worker 가 실제 CPU 를 얼마나 활용했는지 측정해야 한다. 넷째, 성능 저하가 발생한 경우 synchronization, IPC, merge, queue overhead 중 어떤 원인이 있는지 분석해야 한다.

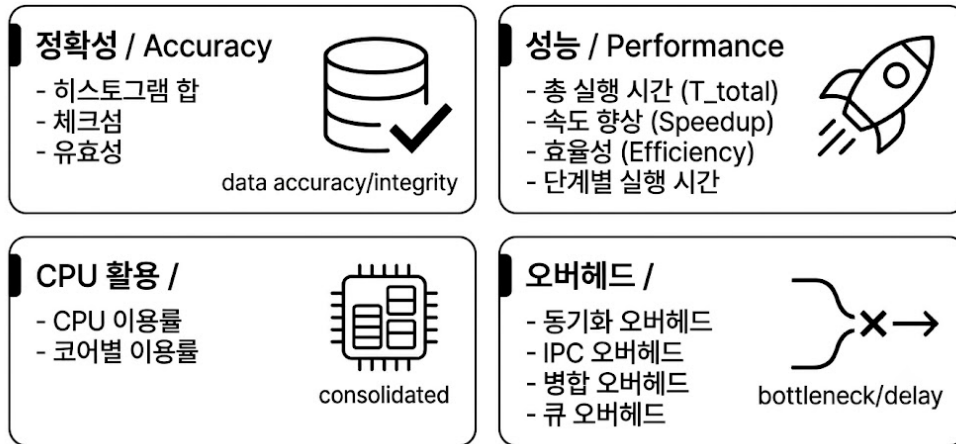


그림 2. 최종 평가 지표 체계

1-9. 시스템 개요 요약

본 프로젝트는 Monte Carlo 기반 CPU-bound workload 를 이용해 운영체제 병렬 처리 구조를 비교하는 실험 시스템이다. Thread 구조에서는 synchronization 문제와 reduce 해결 방식을 분석하고, process 구조에서는 child process 와 IPC 를 분석한다. Hybrid 구조는 process 와 thread 의 계층적 결합을 보여주며, pipeline 구조는 producer-consumer 방식과 queue synchronization overhead 를 분석한다.

최종 성능 평가는 정확성이 검증된 mode 만 대상으로 수행한다. 또한 CPU utilization 과 speedup/efficiency 를 분리하여 해석한다. CPU utilization 은 worker 가 CPU 를 얼마나 바쁘게 사용했는지를 보여주고, speedup 과 efficiency 는 그 사용이 실제 실행 시간 감소로 얼마나 이어졌는지를 보여준다.

따라서 본 프로젝트의 핵심은 특정 구조 하나가 항상 가장 빠르다는 결론이 아니라, 동일한 CPU-bound workload 라도 운영체제 실행 구조에 따라 정확성, overhead, scalability 가 달라진다는 점을 실험적으로 확인하는 것이다.

2. 프로그램 구성

2-1. Trial 단위 계산 구조

본 프로젝트의 시뮬레이션은 각 Trial 이 독립적으로 실행되는 구조를 가진다. 각 Trial 은 고유한 Trial index 를 바탕으로 deterministic seed 를 생성한다. 생성된 seed 를 통해 차량의 속도, 거리, 반응시간, 감속력 등 초기 차량 상태를 구성한다. 이후 time step 별로 위치와 속도를 갱신하며 최소 TTC (Time-To-Collision) 및 충돌 여부를 계산하고, 그 결과를 risk histogram 에 1 회 반영하는 흐름으로 진행된다.

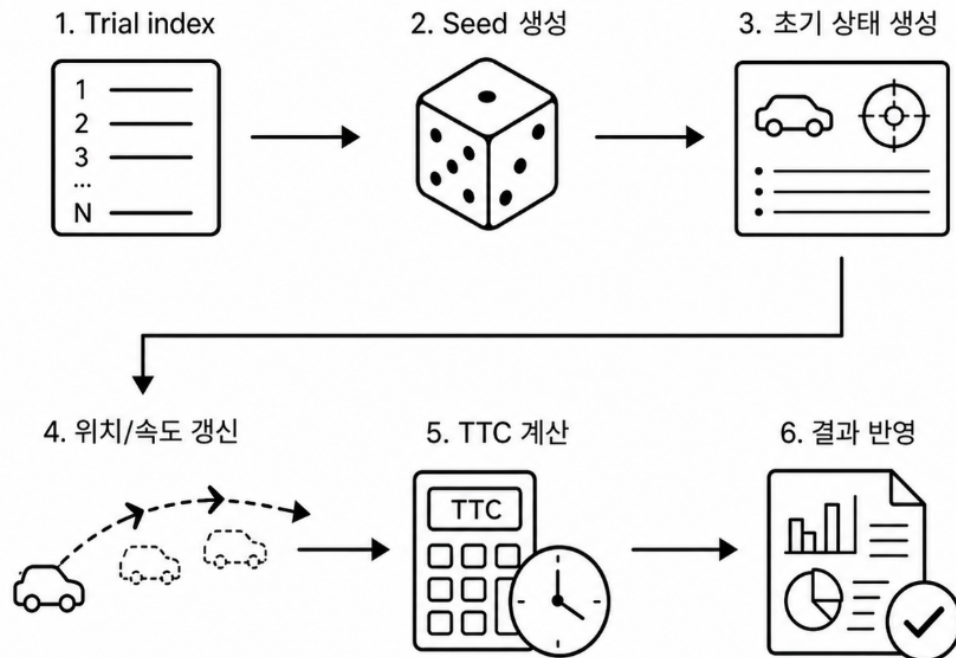


그림 3. Monte Carlo trial 내부 계산 흐름도

2-2. Deterministic Checksum 설계

모든 Trial 은 독립적인 index 기반 seed 를 사용하므로, 실행에 참여하는 worker 수가 달라지더라도 동일한 결과를 생성한다. 정상적으로 동작하는 모드라면 실행 순서나 운영체제 병렬 구조 (Process/Thread) 가 다르더라도 최종적으로 동일한 checksum 이 도출된다. 따라서 병렬 처리의 정확성 기준은 결과 데이터가 손상되지 않았음을 뜻하는 valid=1, 총 수행 횟수를 의미하는 hist_sum=trials, 그리고 순차 실행과 동일한 checksum 일치 여부로 정의한다.

2-3. Workload 조건

실행 구조별 특성을 비교하기 위해 다양한 workload 조건을 구성하여 실험을 진행한다.

표 4. Workload 조건 및 목적

Workload / Profile	목적	해석
default	최종 scaling 기준	실제 Monte Carlo 기본 조건
uniform	균등 workload	static partition 확인
skewed	불균등 workload	queue scheduling 효과 확인
process_friendly	큰 독립 작업	Process coarse-grained 병렬화 확인
thread_friendly	낮은 통신 비용 강조	Thread local reduce 확인

2-4. Thread / Synchronization Mode

Thread mode 는 하나의 process 내부에서 여러 pthread 를 생성하여 Monte Carlo trial 계산을 병렬로 수행하는 구조이다. 각 thread 는 자신에게 할당된 trial range 를 계산하고, 계산 결과를 최종 Result 에 반영한다. Thread 는 process 와 달리 같은 주소 공간을 공유하므로 생성 비용이 비교적 작고 데이터 접근이 빠르다는 장점이 있다. 그러나 여러 thread 가 같은 shared result 에 동시에 접근할 수 있기 때문에 synchronization 처리가 필요하다.

여러 thread 가 같은 Result 구조체에 접근하면 total_trials, collision_count, histogram, checksum 과 같은 공유 데이터가 동시에 갱신될 수 있다. 이때 각 thread 가 같은 값을 읽고 수정한 뒤 다시 저장하는 read-modify-write 과정이 겹치면 일부 update 가 손실되는 lost update 가 발생할 수 있다. 그 결과 hist_sum 이 전체 trials 수와 일치하지 않거나 checksum mismatch 가 발생할 수 있으며, 최종적으로 valid=0 으로 검증될 수 있다. 따라서 thread mode 에서는 단순히 trial 계산을 병렬화하는 것뿐만 아니라, 공유 결과를 안전하게 갱신하기 위한 synchronization 방식이 필요하다.

Race condition 발생 흐름은 다음과 같이 정리할 수 있다.

여러 thread 가 같은 Result 구조체 접근
 → 동시에 read-modify-write 발생
 → lost update 발생
 → hist_sum / checksum 불일치
 → valid=0

본 프로젝트에서는 thread 기반 병렬 처리에서 발생할 수 있는 synchronization 문제를 확인하기 위해 nosync, mutex, reduce 세 가지 방식을 비교하였다. 세 방식은 shared result 를 갱신하는 방식과 정확성 보장 방법에서 차이가 있다.

표 5. nosync / mutex / reduce 방식 비교

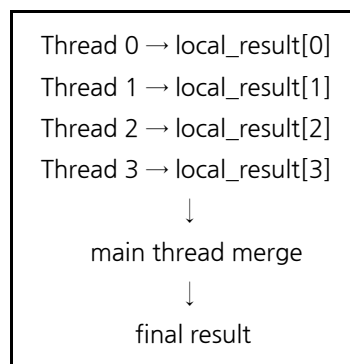
방식	구조	정확성	성능 특성	프로젝트 의미
nosync	shared result 직접 갱신	깨질 수 있음	빠르게 보일 수 있음	race condition 확인용 실패 사례
mutex	critical section 을 lock 으로 보호	정확함	lock contention 발생 가능	synchronization 으로 정확성 보장
reduce	thread-local result 후 merge	정확함	lock overhead 감소	정확성과 성능을 함께 고려한 개선 방식

nosync 는 별도의 동기화 없이 여러 thread 가 shared result 를 직접 갱신하는 방식이다. 이 방식은 lock 을 사용하지 않기 때문에 실행 시간이 빠르게 보일 수 있지만, valid=0 이나 checksum mismatch 가 발생하면 올바른 결과로 사용할 수 없다. 따라서 nosync 는 최적화 방식이 아니라, synchronization 이 없을 때 race condition 이 발생할 수 있음을 보여주는 실패 사례로 해석한다.

mutex 방식은 shared result 를 갱신하는 critical section 을 pthread_mutex_lock 과 pthread_mutex_unlock 으로 보호하는 방식이다. 하나의 thread 만 공유 결과를 수정할 수 있도록 제한하기 때문에 race condition 을 방지하고 정확성을 보장할 수 있다. 하지만 매 trial 마다 lock/unlock 이 반복되면 여러 thread 가 같은 lock 을 기다리는 lock contention 이 발생한다. 이 경우 CPU 가 실제 계산보다 lock 대기 시간을 더 쓰게 되어 synchronization overhead 가 증가할 수 있다.

reduce 방식은 각 thread 가 shared result 를 직접 갱신하지 않고, thread-local result 에 자신이 맡은 trial 결과를 저장하는 방식이다. 모든 thread 의 계산이 끝난 뒤 main thread 가 각 local result 를 한 번에 merge 하여 final result 를 만든다. 이 구조는 hot loop 안에서 shared result 접근을 제거하므로 lock contention 을 줄일 수 있고, 동시에 최종 결과의 정확성도 유지할 수 있다.

Reduce 방식의 흐름은 다음과 같다.



따라서 Thread / Synchronization mode 는 단순히 여러 thread 를 사용해 실행 시간을 줄이는 구조가 아니라, shared result 갱신 방식에 따라 정확성과 성능이 어떻게 달라지는지 확인하기 위한 실험 구조이다. nosync 는 synchronization 이 없을 때 race condition 으로 인해 결과가 깨질 수 있음을 보여주고, mutex 는 정확성을 보장하지만 lock contention 으로 인한 overhead 가 발생할 수 있음을 보여준다. reduce 는 thread-

local result 를 사용하여 공유 데이터 접근을 최소화함으로써 정확성을 유지하면서 synchronization overhead 를 줄이는 방식이다.

2.5. Process Mode

Process mode 는 운영체제의 child process 개념을 반영하기 위해 구현한 실행 구조이다. Thread 는 하나의 process 내부에서 메모리를 공유하지만, process 는 독립된 주소 공간을 가지므로 child 가 계산한 결과가 parent 메모리에 바로 반영되지 않는다. 따라서 process mode 에서는 parent 가 fork()로 child process 를 생성하고, 각 child 가 자신에게 할당된 trial range 를 계산한 뒤 IPC 를 통해 결과를 parent 에게 전달한다.

Process mode 의 실행 흐름은 다음과 같다.

Parent process

- fork()
- Child process 생성
- child 별 trial range 계산
- child-local Result 생성
- pipe 또는 shared memory IPC 로 parent 에 전달
- parent waitpid()
- parent result merge
- checksum / valid 검증

표 6. Process Mode 주요 OS 개념

OS 개념	프로젝트 적용
fork()	child process 생성
child process	독립 trial range 계산
address space	parent 와 child 메모리 분리
IPC	child result 전달
waitpid()	child 종료 회수
merge	parent 가 최종 result 생성

2.6. IPC 구조

Process 는 독립된 주소 공간을 사용하기 때문에 child process 의 계산 결과를 parent 가 직접 접근할 수 없다. 이를 해결하기 위해 본 프로젝트에서는 pipe IPC 와 shared memory IPC 를 사용하였다.

Pipe IPC 는 child 가 계산한 Result 를 kernel buffer 에 write()하고, parent 가 이를 read()하여 병합하는 방식이다. 구조가 단순하고 parent-child 간 데이터 전달 흐름이 명확하지만, read/write 과정에서 overhead 가 발생한다.

Shared memory IPC 는 anonymous mmap 기반 shared result table 을 만들고, parent 와 child 가 함께 접근할 수 있는 result slot 을 제공하는 방식이다. 각 child 는 자신에게 할당된 독립 slot 에만 결과를 저장하고, parent 는 waitpid() 이후 해당 slot 을 읽어 최종 결과에 병합한다.

표 7. Pipe IPC 와 Shared Memory IPC 비교

항목	Pipe IPC	Shared Memory IPC
전달 방식	kernel buffer read/write	shared result slot
구조	단순	slot 설계 필요
복사 비용	있음	상대적으로 적음
동기화	read/write 순서 관리	write/read 순서 관리
의미	IPC overhead 관찰	process-friendly 최적화
주의점	descriptor 관리 필요	slot 충돌 방지 필요

2.7. Hybrid Mode

Hybrid mode 는 process 와 thread 를 계층적으로 결합한 구조이다. Parent process 는 child process 를 생성하고, 각 child process 내부에서는 다시 pthread worker 를 생성하여 trial 계산을 병렬로 수행한다.

즉, process 는 큰 작업 그룹을 분할하고, thread 는 child 내부의 세부 trial range 를 병렬처리한다. 각 thread 의 결과는 child 내부에서 먼저 병합되고, child-local Result 는 pipe 또는 shared memory 를 통해 parent 에게 전달된다.

Hybrid mode 의 흐름은 다음과 같다.

Parent process

- child process 생성
- child 내부 pthread worker 생성
- thread-local reduce
- child-local Result 생성
- IPC 로 parent 에 전달
- parent final merge

Hybrid 구조는 process isolation 과 thread 의 lightweight 병렬성을 함께 사용할 수 있다는 장점이 있다. 그러나 child process 생성, pthread 생성 및 join, thread-local merge, IPC 비용이 모두 포함되므로 구조가 복잡하고 overhead 도 함께 증가할 수 있다.

2.8. Process / Thread / Hybrid 구조 비교

표 8. Process / Thread / Hybrid 구조 비교

구조	장점	단점	적합 상황
Thread	생성 비용 낮음, 메모리 공유 쉬움	Race condition 가능	같은 주소 공간 내 세부 계산
Process	독립 주소 공간, 안정성	fork/IPC/waitpid overhead	큰 독립 작업
Hybrid	계층적 병렬화 가능	구조 복잡, overhead 복잡	큰 작업 + 내부 병렬화

2.9. Pipeline Mode

Pipeline mode 는 Monte Carlo trial 계산을 batch 단위로 나누고, 이를 queue 를 통해 worker thread 에게 전달하는 producer-consumer 구조이다. 일반적인 thread mode 가 thread 별로 고정된 trial range 를 나누어 계산하는 방식이라면, pipeline mode 는 전체 작업을 여러 batch 로 분리한 뒤 TaskQueue 에 저장하고, worker 들이 queue 에서 작업을 하나씩 가져가 처리한다. 이를 통해 작업 생성, 계산, 결과 전달, 병합 과정을 단계별로 나누어 분석할 수 있다.

Pipeline mode 의 전체 구조는 다음과 같다.

```

Preprocessor
→ TaskQueue
→ Worker Pool
→ MergeQueue
→ Aggregator
→ Final Result
  
```

표 9. Pipeline mode 구성요소와 역할

구성요소	역할
Preprocessor	Monte Carlo trial 을 batch 단위로 생성
TaskQueue	생성된 batch 를 저장하고 worker 에게 전달
Worker Pool	TaskQueue 에서 batch 를 가져와 Monte Carlo 계산 수행
MergeQueue	worker 가 계산한 partial result 를 전달
Aggregator	MergeQueue 에서 partial result 를 꺼내 병합
Final Result	최종 결과 검증 대상

Preprocessor 는 계산할 trial 들을 batch 단위로 생성하여 TaskQueue 에 넣는다. Worker Pool 의 각 worker thread 는 TaskQueue 에서 batch 를 꺼내 Monte Carlo 계산을 수행하고, 계산이 끝나면 partial result 를 생성한다. 이후 병합 방식에 따라 worker 의 partial result 는 마지막에 한 번에 병합되거나, MergeQueue 를 통해 Aggregator 에게 전달된다. 최종적으로 병합된 결과는 hist_sum, checksum, valid 값을 통해 sequential baseline 과 동일한 결과를 유지하는지 검증한다.

Pipeline mode 에서 TaskQueue 와 MergeQueue 는 여러 thread 가 함께 접근하는 공유 queue 이다. TaskQueue 는 producer 가 생성한 batch 를 worker 에게 전달하는 역할을 하고, MergeQueue 는 worker 가 계산한 partial result 를 aggregator 에게 전달하는 역할을 한다. 두 queue 모두 내부적으로 buffer, head, tail, count, closed 와 같은 공유 상태를 가지기 때문에 여러 thread 가 동시에 접근하면 queue 상태가 깨질 수 있다. 따라서 queue 내부 상태를 수정하는 구간은 mutex 로 보호하고, queue 가 비어 있거나 가득 찬 상황에서는 condition variable 을 사용하여 thread 를 대기시키거나 다시 깨운다.

표 10. TaskQueue / MergeQueue 동기화 문제와 해결 방식

문제 상황	해결 방식
queue 상태 동시 접근	mutex 로 queue 내부 상태 보호
queue empty 상태	not_empty condition variable 사용
queue full 상태	not_full condition variable 사용
busy waiting 발생 가능성	pthread_cond_wait / pthread_cond_signal 사용

즉, mutex 는 공유 queue 의 상태를 안전하게 보호하기 위한 장치이고, condition variable 은 empty/full 상태에 따라 thread 를 효율적으로 대기시키고 깨우기 위한 장치이다. queue 가 비어 있으면 consumer thread 는 not_empty 조건에서 대기하고, 새로운 작업이나 결과가 들어오면 signal 을 통해 다시 실행된다. 반대로 queue 가 가득 차 있으면 producer thread 는 not_full 조건에서 대기하고, 공간이 생기면 다시 실행된다. 이 구조를 통해 불필요하게 queue 상태를 반복 확인하는 busy waiting 을 줄이고, producer-consumer 구조를 안정적으로 구현할 수 있다.

Pipeline mode에서는 결과 병합 방식으로 Final Reduce 와 Interactive Merge 를 비교하였다. Final Reduce는 모든 worker가 계산을 끝낸 뒤 마지막에 partial result를 한 번에 병합하는 방식이다. 구조가 단순하고 중간 병합을 위한 별도의 aggregator thread나 merge queue 동작이 필요하지 않기 때문에 synchronization overhead가 비교적 적다. 반면 Interactive Merge는 worker가 batch 계산을 끝낼 때마다 partial result를 MergeQueue에 전달하고, 별도의 aggregator thread가 이를 꺼내 실행 중간에 계속 병합하는 방식이다.

표 11. Final Reduce 와 Interactive Merge 구조 비교

항목	Final Reduce	Interactive Merge
병합 시점	모든 worker 종료 후	실행 중간
구조	마지막에 한 번 병합	MergeQueue + Aggregator
장점	구조가 단순하고 동기화 비용이 작음	지속 집계와 monitoring이 필요한 구조 분석
단점	실행 중 중간 결과를 확인하기 어려움	지속 집계와 monitoring이 필요한 구조 분석
프로젝트 의미	최종 결과만 필요한 batch 작업에 적합	지속 집계와 monitoring이 필요한 구조 분석

Interactive Merge는 producer-consumer 흐름과 중간 결과 병합 과정을 보여줄 수 있다는 장점이 있지만, MergeQueue 접근, mutex lock/unlock, condition variable wait/signal, aggregator thread 실행 비용이 추가된다. 따라서 Pipeline mode는 단순히 실행 시간을 줄이기 위한 구조라기보다, queue 기반 작업 분배와 synchronization overhead를 분석하기 위한 구조이다. Final Reduce는 단순하고 안정적인 병합 기준으로 사용할 수 있고, Interactive Merge는 실행 중간 병합이 가능한 현실적인 pipeline 구조를 보여준다. Interactive Merge가 Final Reduce보다 느리게 측정되더라도 실패로 해석하지 않고, 중간 결과 집계 기능을 얻기 위해 queue와 condition variable, aggregator thread 비용을 지불하는 구조적 trade-off로 해석한다.

3. 동작 검증 및 Test Vector 설계

3.1 Sequential Mode

Sequential mode 는 단일 process 내부에서 단일 실행 흐름으로 모든 Monte Carlo trial 을 순차적으로 처리하는 방식이다. 이 모드의 실행 시간은 병렬 실행 구조의 성능 향상을 평가하는 속도 향상(speedup) 및 효율성(efficiency) 계산의 기준값(baseline)으로 사용된다. Speedup 은 $T_{seq} / T_{parallel}$ 로 계산되며, Efficiency 는 해당 speedup 을 worker 수로 나눈 값으로 도출한다.

3-2. Thread / Synchronization Test Vector

Thread / Synchronization Test Vector 는 thread 기반 병렬 처리에서 shared result 갱신 방식에 따라 정확성과 성능이 어떻게 달라지는지 검증하기 위해 구성하였다. Thread mode 에서는 여러 pthread 가 같은 주소 공간을 공유하므로 histogram, collision_count, checksum 과 같은 결과 데이터에 동시에 접근할 수 있다. 따라서 단순히 실행 시간이 짧은지만 확인하는 것이 아니라, 각 방식이 sequential baseline 과 동일한 결과를 유지하는지 함께 확인해야 한다.

검증 기준은 hist_sum == trials, checksum 일치 여부, valid 값으로 설정하였다. hist_sum 이 trials 와 일치하면 모든 trial 이 누락 없이 계산되었다고 볼 수 있고, checksum 이 sequential baseline 과 일치하면 실행 구조가 달라도 동일한 계산 결과가 유지되었음을 의미한다. valid 값은 이러한 조건을 종합하여 정상 실행 여부를 판단하는 기준으로 사용하였다.

본 프로젝트에서는 synchronization 방식에 따라 nosync, mutex, reduce 를 test vector 로 구성하였다. nosync 는 synchronization 없이 여러 thread 가 shared result 를 직접 갱신하는 조건이다. 이 test vector 의 목적은 정확한 결과를 얻는 것이 아니라, 동기화가 없을 때 shared result 갱신 과정에서 race condition 이 발생할 수 있음을 확인하는 것이다. 따라서 nosync 가 빠르게 보이더라도 valid=0 이나 checksum mismatch 가 발생하면 실패 사례로 해석한다.

mutex 는 shared result 를 갱신하는 critical section 을 lock 으로 보호하는 조건이다. 이 test vector 에서는 lock 을 사용했을 때 race condition 이 방지되고, valid 와 checksum 이 정상적으로 유지되는지 확인한다. 다만 mutex 는 정확성을 보장하는 대신 매 trial 마다 lock/unlock 이 발생할 수 있으므로, lock contention 으로 인한 synchronization overhead 도 함께 확인해야 한다.

reduce 는 각 thread 가 shared result 를 직접 수정하지 않고 thread-local result 에 계산 결과를 저장한 뒤, 모든 thread 가 종료된 후 main thread 가 한 번에 병합하는 조건이다. 이 test vector 에서는 shared result 접근을 줄였을 때도 최종 결과가 sequential baseline 과 일치하는지 확인한다. reduce 방식은 hot loop 에서 shared write 를 제거하기 때문에 race condition 을 방지하면서 mutex 방식의 lock contention 을 줄일 수 있는 구조로 검증하였다.

표 12. Thread / Synchronization Test Vector 검증 조건

Test Vector	목적	검증 기준	결과 해석
nosync	race condition 발생 여부 확인	hist_sum, checksum, valid	valid=0 또는 checksum mismatch 발생 시 synchronization 필요성 확인
mutex	lock 기반 정확성 확인	hist_sum == trials, checksum match, valid=1	정확성은 회복되지만 lock contention overhead 발생 가능
reduce	thread-local result 기반 개선 확인	hist_sum == trials, checksum match, valid=1	shared write 를 줄여 정확성과 성능을 함께 확보

이 test vector 의 목적은 어떤 방식이 항상 가장 빠른지를 판단하는 것이 아니라, thread 병렬 처리에서 정확성 보장과 synchronization overhead 사이의 trade-off 를 확인하는 것이다. nosync 는 의도적으로 안전하지 않은 비교군이며, mutex 와 reduce 는 결과 정확성을 회복하기 위한 해결 방식이다. 최종 분석에서는 실행 시간뿐만 아니라 valid 와 checksum 결과를 함께 기준으로 사용한다.

3-3. Process / Hybrid Test Vector

Process / IPC / Hybrid 구조의 정상 동작을 검증하기 위해 process 수와 process-thread 조합을 변화시키며 실험하였다. 검증 기준은 hist_sum 이 trials 와 일치하는지, valid 값이 1 인지, checksum 이 sequential 기준과 일치하는지이다.

표 13. Process / Hybrid Test Vector

Test Vector	목적	실행 조건	기대 결과
Process 1	단일 child process overhead 확인	process=1	valid=1, hist_sum=trials
Process 2	multi-process 병렬화 확인	process=2	valid=1, hist_sum=trials
Process 4	process scaling 확인	process=4	valid=1, hist_sum=trials
Hybrid 2x2	process + thread 결합 검증	process=2, thread=2	valid=1, hist_sum=trials
Hybrid 2x4	child 내부 thread 증가 효과 확인	process=2, thread=4	valid=1, hist_sum=trials
Hybrid 4x2	process 수 증가와 내부 thread 결합 확인	process=4, thread=2	valid=1, hist_sum=trials
Process pipe/shm	IPC 방식별 결과 전달 비교	process=4, ipc=pipe/shm	valid=1, checksum match

특히 process mode에서는 pipe와 shared memory IPC를 모두 확인하여, child result가 parent에게 정상적으로 전달되는지와 IPC 방식에 따른 차이를 함께 검증하였다.

3-4. Pipeline Test Vector

Pipeline Test Vector는 queue 기반 작업 분배와 결과 병합 방식이 정상적으로 동작하는지 검증하기 위해 구성하였다. Pipeline mode에서는 Monte Carlo trial이 batch 단위로 나뉘고, worker thread가 TaskQueue에서 batch를 가져가 계산한 뒤 partial result를 생성한다. 따라서 이 test vector에서는 단순히 최종 계산 결과만 확인하는 것이 아니라, batch가 누락 없이 worker에게 전달되는지, worker가 계산한 partial result가 정상적으로 병합되는지, 그리고 최종 결과가 sequential baseline과 일치하는지를 함께 확인하였다.

검증 기준은 $hist_sum == trials$, checksum 일치 여부, valid 값으로 설정하였다. $hist_sum$ 이 trials와 일치하면 모든 trial이 누락 없이 계산되었다고 볼 수 있고, checksum이 sequential baseline과 일치하면 mode가 달라도 동일한 계산 결과가 유지되었음을 의미한다. valid 값은 이러한 검증 조건을 종합하여 정상 실행 여부를 판단하는 기준으로 사용하였다.

본 프로젝트에서는 Pipeline mode의 병합 방식을 Final Reduce와 Interactive Merge로 나누어 검증하였다. Final Reduce test vector는 모든 worker가 계산을 끝낸 뒤 마지막에 partial result를 한 번에 병합하는 조건이다. 이 조건에서는 TaskQueue를 통해 batch가 정상적으로 분배되고, worker별 partial result가 누락 없이 최종 결과에 반영되는지를 확인한다. Final Reduce는 구조가 단순하기 때문에 Pipeline mode의 기본 비교 기준으로 사용하였다.

Interactive Merge test vector는 worker가 계산한 partial result를 실행 중간에 MergeQueue로 전달하고, aggregator thread가 이를 꺼내 최종 Result에 병합하는 조건이다. 이 조건에서는 계산 결과의 정확성뿐만 아니라, MergeQueue를 사용하는 producer-consumer 병합 과정이 안전하게 동작하는지도 확인한다. worker와 aggregator가 같은 MergeQueue를 공유하므로 queue의 buffer, head, tail, count와 같은 상태가 mutex로 보호되어야 하며, empty/full 상태에서는 condition variable을 통해 thread가 대기하고 다시 깨어나야 한다.

Pipeline Test Vector에서 발생할 수 있는 주요 문제는 batch 또는 partial result가 누락되거나, queue 상태가 잘못 변경되어 최종 결과가 sequential baseline과 달라지는 것이다. 이를 방지하기 위해 TaskQueue와 MergeQueue에서는 mutex로 queue 내부 상태를 보호하고, not_empty와 not_full condition variable을 사용하여 empty/full 상태를 처리하였다. 이 기준을 만족하면 batch 분배와 partial result 병합이 정상적으로 수행된 것으로 판단하였다.

Pipeline Test Vector는 다음 조건을 중심으로 검증하였다.

표 14. Pipeline Test Vector 검증 조건

Test Vector	목적	검증 기준	결과 해석
Pipeline Final Reduce	batch 분배와 최종 병합 정상 동작 확인	hist_sum == trials, checksum match, valid=1	worker 별 partial result 가 누락 없이 최종 결과에 반영됨
Pipeline Interactive Merge	MergeQueue 와 Aggregator 기반 중간 병합 확인	hist_sum == trials, checksum match, valid=1	producer-consumer 병합 과정이 정상 수행됨
Queue scheduling 조건	batch 단위 동적 작업 분배 확인	실행 시간, valid, checksum	queue overhead 와 load balancing 영향을 함께 분석

따라서 Pipeline Test Vector 의 목적은 어떤 병합 방식이 항상 더 빠르다는 결론을 내기 위한 것이 아니라, Final Reduce 와 Interactive Merge 가 모두 정확성 기준을 만족하는지 확인하고, 병합 방식에 따라 발생할 수 있는 overhead 를 이후 성능 분석에서 해석하기 위한 것이다. Final Reduce 는 단순하고 안정적인 병합 기준으로 사용되며, Interactive Merge 는 실행 중간 병합이 가능한 producer-consumer 구조를 검증하는 조건이다. Interactive Merge 가 느리게 측정되더라도 실패로 해석하지 않고, queue 와 condition variable, aggregator thread 를 사용하는 pipeline 구조의 synchronization overhead 로 해석한다.

4. 성능 측정 방법

4-1. 실험 환경

표 15. 실험 환경 및 측정 도구

항목	값
실행 환경	Docker Desktop 기반 Ubuntu 22.04 Linux VM
Architecture	aarch64
Docker 할당 CPU	10
Compiler	GCC. -O2, POSIX pthread
CPU 측정	GNU /user/bin/time -v
Core 별 측정	mpstat -P ALL 1
정확성 기준	Valid, hist_sum, checksum

단, Docker Desktop 은 Linux VM 위에서 실행되므로 해당 환경 내부에서의 상대적인 성능 비교에는 유효하나, 이를 native Linux 환경의 절대적인 성능 수치로 일반화할 수는 없다.

4-2. Main Strong Scaling 조건

총 Workload 의 크기를 고정하고 Worker 수만을 변화시키는 Strong Scaling 방식으로 성능을 측정한다.

표 16. Strong Scaling 실험 조건 및 검증 체계

항목	값
Trials	120,000,000
Steps	50
Profile	default
Pre/Post work	0 / 0
Worker 수	1, 2, 4, 8
반복	각 case 5 회
Affinity	활성화

4-3. 정확성 판정 기준

성능 비교 데이터로 사용되기 위해서는 다음 조건을 모두 충족해야 한다.

- valid = 1
- hist_sum = trials
- checksum = sequential checksum
- 반복 실행 모두 checksum_count = 1

4-4. 측정 지표

표 17. 주요 성능 측정 지표

Metric	의미
time_total	end-to-end wall-clock 시간
time_compute	worker 실행 구간
speedup	기준 시간 대비 단축 비율 ($T_{seq} / T_{parallel}$)
efficiency	worker 당 speedup 효율 (speedup / workers)
CPU%	전체 worker CPU 활용률
Util/Worker	CPU% / worker 수
Max RSS	GNU time 이 관측한 최대 resident memory

time_sync 는 여러 worker 의 대기 시간 누적값이므로 모든 stage 값을 더해도 time_total 과 일치하지 않을 수 있다.

5. 실험 결과 및 분석

5-1. Strong Scaling 결과 분석

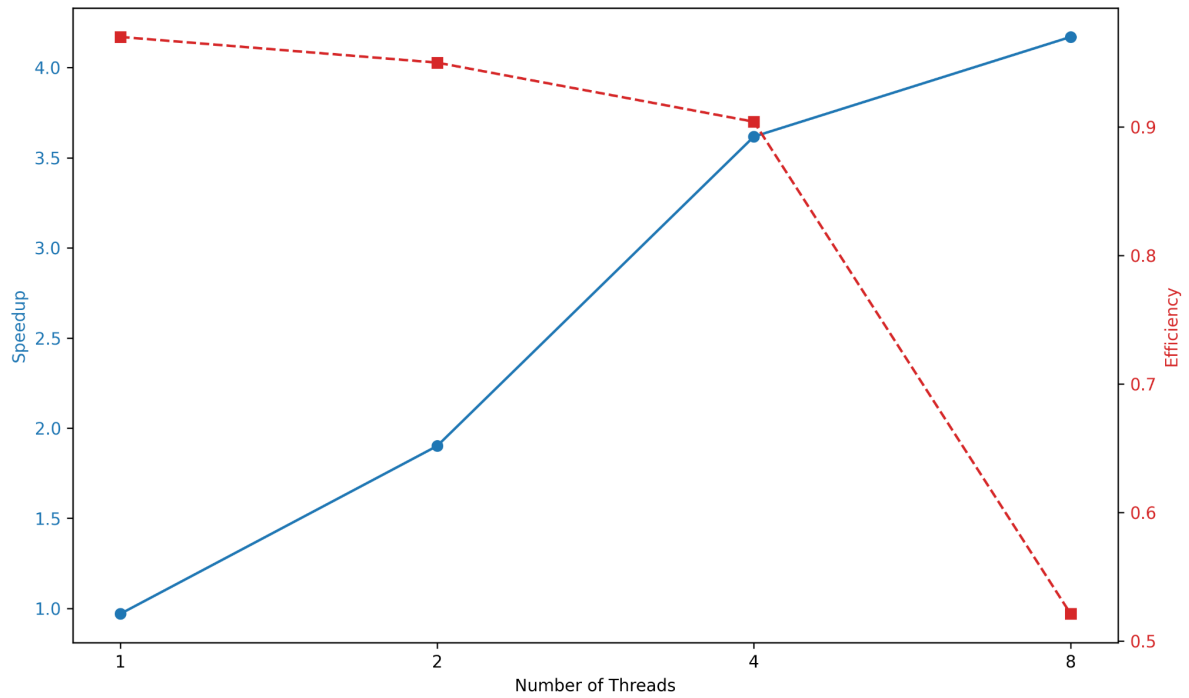


그림 4. Thread Mode 의 Worker 수에 따른 Speedup 및 Efficiency 변화

(파란색 실선: 기준 시간 대비 속도 향상 비율, 빨간색 점선: Worker 당 효율성. 4-worker 이후 Efficiency 가 급감하는 것을 확인할 수 있다.)

표 18. 실제 Monte Carlo Strong Scaling 결과

Case	Workers	Avg Time	Stdev	Speedup	Efficiency	CPU/Worker
seq	1	12.111s	0.366s	1.000x	100.00%	99.0%
thread_2_reduce	2	6.148s	0.173s	1.970x	98.50%	99.5%
thread_4_reduce	4	3.236s	0.016s	3.743x	93.60%	99.7%
process_4_shm	4	3.222s	0.014s	3.759x	94.00%	99.5%
hybrid_2x2_shm	4	3.222s	0.031s	3.758x	94.00%	99.5%
pipeline_4_final	4	3.275s	0.137s	3.698x	92.50%	99.8%
thread_8_reduce	8	2.390s	0.045s	5.068x	63.30%	95.7%
pipeline_8_final	8	2.177s	0.058s	5.564x	69.60%	99.7%

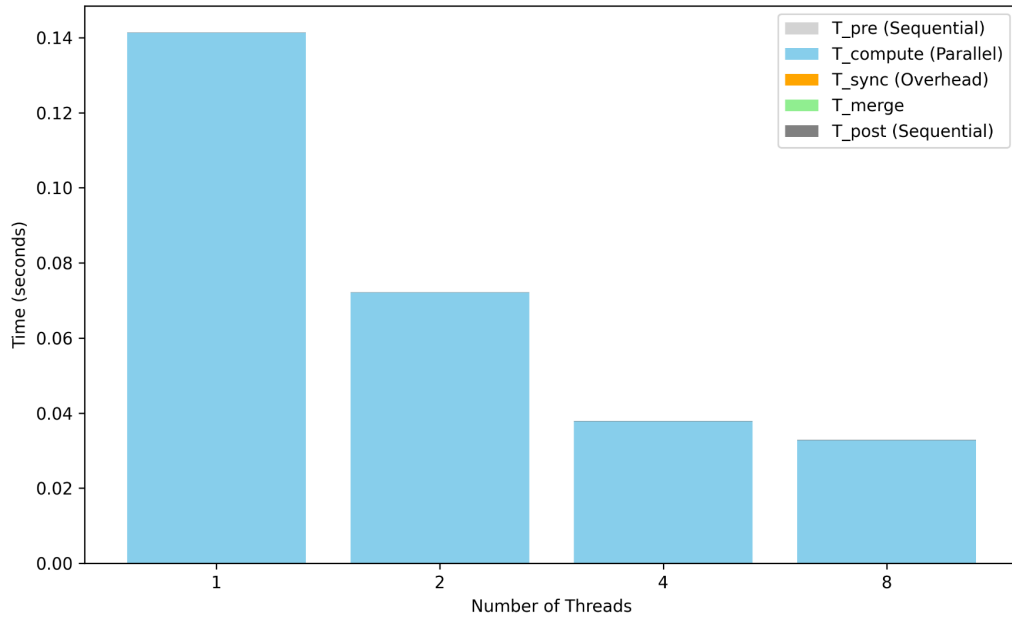


그림 5. Thread Mode 의 Worker 수에 따른 단계별 실행 시간(Stage Time) 변화

(하늘색 막대: 실제 병렬 연산 구간인 T_compute. Worker 수가 증가할수록 연산 시간이 단축되지만, 4-worker 에서 8-worker 로 넘어갈 때는 단축 폭이 크게 둔화되는 것을 확인할 수 있다.)

실험 결과 2-worker 환경은 거의 이상적인 확장성($T1/2$)에 도달하였다. 4-worker 환경 역시 주요 실행 구조(thread, process, hybrid, pipeline) 모두 실제 Monte Carlo workload 에서 92.5~94.0%의 efficiency를 달성하며 N-core scaling에 상당히 근접한 결과를 보여주었다. 특히 큰 workload를 처리할 때 Process 구조나 Hybrid 구조 역시 Thread 구조와 충분히 경쟁 가능한 성능을 보였다. 반면 8-worker 환경에서는 CPU 자원을 계속 할당받아 사용함에도 불구하고 efficiency가 감소하였다. 이는 코어를 바쁘게 유지하여 CPU utilization을 높이는 것과, 동기화 및 IPC 오버헤드 없이 실제 실행 시간(speedup)이 선형으로 단축되는 것은 동일한 지표가 아님을 시사한다.

5-2. Core 별 CPU Utilization

표 19. 4-Worker 구조 Core 별 CPU 활용률

Case	CPU%	Util/Worker	Core 별 평균	Min Core	Stdev	Max RSS
thread_4_reduce	398%	99.50%	100.0 100.0 100.0 100.0	100.00%	0	1,292 KB
process_4_shm	398%	99.50%	100.0 100.0 100.0 100.0	100.00%	0	1,212 KB
hybrid_2x2_shm	397%	99.20%	100.0 99.7 100.0 100.0	99.70%	0.16	1,220 KB
pipeline_4_final	399%	99.80%	100.0 100.0 100.0 100.0	100.00%	0	1,472 KB

4-worker 대표 구조 실험에서 worker 당 평균 CPU 활용률은 99.2~99.8%로 측정되었다. 대상 core들이 거의 균일하게 100%에 근접하게 사용되고 있어, 시스템이 할당된 코어 자원을 효율적으로 포화 상태(saturation)로 끌어올림을 확인하였다.

5.3 Process-Friendly / Thread-Friendly 교차 비교

표 20. Workload Profile 교차 비교

Profile / Case	Workers	Avg Time	Speedup	Efficiency
process-friendly seq	1	0.8620s	1.000x	100.00%
process-friendly process 4 shm	4	0.2404s	3.586x	89.60%
process-friendly thread 4 reduce	4	0.2412s	3.573x	89.30%
thread-friendly seq	1	0.0338s	1.000x	100.00%
thread-friendly thread 4 reduce	4	0.0098s	3.431x	85.80%
thread-friendly process 4 shm	4	0.0104s	3.236x	80.90%

단일 구조가 모든 환경에서 우월한 것은 아니다. process-friendly 조건에서는 작업 크기가 충분히 커서 process, thread, hybrid 구조 간의 성능 차이가 미미해진다. 반면 thread-friendly 조건에서는 낮은 통신 비용 덕분에 thread-local reduce 를 활용한 방식이 유리하게 측정되었다. 이는 workload 의 특징에 따라 최적의 운영체제 실행 구조를 선택해야 함을 의미한다.

5-4. Synchronization 결과 분석

Synchronization 결과 분석에서는 shared result 를 갱신하는 방식에 따라 정확성과 성능이 어떻게 달라지는지 확인하였다. 실험은 Docker Desktop 기반 Ubuntu 22.04 Linux VM 환경에서 수행되었으며, 결과는 동일한 환경 안에서 synchronization 방식별 상대 차이를 비교하기 위한 값으로 해석하였다. Docker Desktop 은 Linux VM 위에서 실행되므로, 해당 수치를 native Linux 의 절대 성능으로 일반화하지 않는다.

먼저 nosync 는 shared result 를 보호하지 않고 여러 thread 가 동시에 갱신하는 방식이다. 최신 hybrid synchronization 실험에서 hybrid_2x2_nosync_shm 은 평균 실행 시간 0.0312s 로 측정되었지만, valid 값은 0 이었고 5 회 반복 모두 checksum 이 서로 일치하지 않았다. 실행 시간만 보면 빠르게 보일 수 있지만, 결과 정확성이 깨졌기 때문에 올바른 실행 결과로 사용할 수 없다. 즉, nosync 는 성능 좋은 방식이 아니라 race condition 을 재현하기 위한 실패 사례이다.

표 21. nosync race condition 결과

Case	Avg Time	Valid	Checksum 상태	해석
hybrid_2x2_nosync_shm	0.0312s	0	5 회 모두 불일치	race condition 으로 결과가 깨진 실패 사례

이 결과의 원인은 여러 thread 가 같은 shared result 에 대해 동시에 read-modify-write 를 수행하기 때문이다. 예를 들어 두 thread 가 같은 histogram 값을 읽은 뒤 각각 증가시키면, 실제로는 두 번 증가해야 하는 값이 한 번만 반영될 수 있다. 이러한 lost update 가 누적되면 hist_sum 이 trials 와 일치하지 않거나

checksum 이 sequential baseline 과 달라지고, 최종적으로 valid=0 이 된다. 따라서 synchronization 결과 분석에서는 실행 시간보다 먼저 valid 와 checksum 을 확인해야 한다.

mutex 방식은 shared result 갱신 구간을 lock 으로 보호하여 정확성을 회복한다. 그러나 모든 trial 에서 같은 shared result lock 을 획득해야 하므로 lock contention 이 발생한다. Thread 2-worker 실험에서 thread_2_mutex 는 valid=1 을 유지했지만, 평균 실행 시간은 0.1794s, 평균 sync time 은 0.1077s, lock count 는 1,000,000 으로 나타났다. 반면 thread_2_reduce 는 평균 실행 시간 0.0509s, sync time 은 거의 0s, lock count 는 0 으로 측정되었다. 이는 mutex 방식에서 lock 대기 시간이 전체 성능에 큰 영향을 줄 수 있음을 보여준다.

표 22. mutex 와 reduce 의 lock contention 비교

Case	Avg Time	Avg SyncTime	Lock Count	Valid	해석
thread_2_mutex	0.1794s	0.1077s	1,000,000	1	trial 마다 lock 을 사용하여 lock contention 이 큼
thread_2_reduce	0.0509s	약 0s	1	1	hot loop 에서 shared write 와 lock 을 제거

마지막으로 reduce 방식은 각 worker 가 독립적인 local result 를 계산하고, worker 종료 이후 한 번만 merge 하는 방식이다. Hybrid 2x2 조건에서 hybrid_2x2_mutex_shm 은 평균 실행 시간 0.0563s, speedup 1.866x, efficiency 46.7%로 측정되었다. 반면 hybrid_2x2_reduce_shm 은 평균 실행 시간 0.0288s, speedup 3.643x, efficiency 91.1%로 측정되었고, 두 방식 모두 valid 값은 1 이었다. 같은 정확성을 유지하면서 reduce 방식이 mutex 보다 약 1.95x 빠르게 나타난 것이다.

표 23. Hybrid synchronization 방식별 결과 비교

Case	Avg Time	Speedup	Efficiency	Valid	해석
hybrid_2x2_mutex_shm	0.0563s	1.866x	46.7%	1	정확성은 보장하지만 lock contention overhead 가 큼
hybrid_2x2_reduce_shm	0.0288s	3.643x	91.1%	1	local reduce 로 정확성과 성능을 함께 확보

결과적으로 Synchronization 실험은 단순히 어떤 방식이 가장 빠른지를 확인하기 위한 것이 아니라, 병렬 처리에서 정확성과 synchronization overhead 사이의 trade-off 를 분석하기 위한 실험이다. nosync 는 빠르게 보일 수 있어도 valid=0 과 checksum mismatch 가 발생하므로 사용할 수 없다. mutex 는 정확성을 보장하지만 trial 단위 lock contention 으로 인해 성능 저하가 발생할 수 있다. reduce 는 thread-local result 를 사용하여 hot loop 에서 shared write 를 제거하고, join 이후 한 번에 merge 함으로써 정확성을 유지하면서 synchronization overhead 를 줄이는 방식이다. 따라서 본 프로젝트에서는 nosync 를 race

condition 의 증거로, mutex 를 정확성 회복 방식으로, reduce 를 가장 실용적인 synchronization 해결 방식으로 해석하였다.

5-5. Process / IPC 결과 분석

Process mode 는 child process 를 이용해 Monte Carlo trial range 를 분할 처리한다. 실험 결과 process mode 는 정상 조건에서 valid=1 과 checksum match 를 만족하여, child process 가 계산한 결과가 pipe 또는 shared memory IPC 를 통해 parent process 로 정상 전달되고 최종 merge 가 올바르게 수행됨을 확인하였다.

Process 성능은 workload 크기와 IPC overhead 의 영향을 함께 받는다. process_1 은 sequential 보다 느릴 수 있는데, 이는 실제 병렬화 이득 없이 fork, IPC, waitpid, result merge 비용이 추가되기 때문이다. 반면 process 수가 증가하고 workload 가 충분히 커지면 각 child process 가 독립적으로 계산을 수행하므로 병렬화 이득이 발생할 수 있다.

다만 process 가 항상 thread 보다 빠르다고 해석해서는 안 된다. Thread 는 같은 주소 공간을 공유하기 때문에 결과 전달 비용이 작지만, process 는 독립 주소 공간을 사용하므로 IPC 가 필요하다. 따라서 process mode 의 의미는 단순 성능 우위가 아니라, process isolation 과 IPC overhead 를 실험적으로 확인하는 데 있다.

Pipe IPC 와 shared memory IPC 도 서로 다른 특성을 가진다. Pipe 는 구조가 단순하고 명시적인 read/write 흐름을 보여주지만 kernel buffer 를 통한 전달 비용이 있다. Shared memory 는 copy overhead 를 줄일 수 있지만, child 별 slot 관리와 접근 순서 설계가 필요하다. 본 프로젝트에서는 shared memory mode 에서 child 별 독립 slot 을 사용하고 parent 가 waitpid() 이후 read 하도록 하여 충돌 가능성을 줄였다.

최신 실제 Monte Carlo strong scaling 결과에서 process_4_shm 은 4-worker 조건에서 평균 3.222s, speedup 3.759x, efficiency 94.0%를 기록하였다. 이는 fork 와 IPC overhead 가 존재하더라도 workload 가 충분히 크고 child 별 결과 전달이 1 회로 제한되면 process 구조도 높은 병렬화 효율을 보일 수 있음을 의미한다.

다만 process_4_shm 과 thread_4_reduce 의 평균 시간 차이는 매우 작으므로, process 가 모든 환경에서 thread 보다 빠르다고 일반화해서는 안 된다. 본 결과는 동일 Docker Desktop Linux VM 환경에서 실제 Monte Carlo workload 를 기준으로 한 상대 비교 결과로 해석해야 한다.

5-6. Hybrid 결과 분석

Hybrid mode 는 process 와 thread 를 결합한 계층적 병렬 구조이다. Parent process 는 child process 를 생성하고, 각 child process 내부에서는 pthread worker 들이 trial range 를 나누어 계산한다. 이후 child 내부에서 thread-local result 를 child-local Result 로 병합하고, 이를 pipe 또는 shared memory IPC 를 통해 parent 에게 전달한다.

Hybrid 구조는 process 와 thread 의 역할을 명확히 분리한다. Process 는 큰 작업 그룹을 나누고, thread 는 child process 내부의 세부 계산을 병렬화한다. 이를 통해 process isolation 과 thread 의 lightweight 병렬성을 동시에 사용할 수 있다.

그러나 Hybrid mode 는 구조가 복잡하다. child process 생성 비용, child 내부 thread 생성 및 join 비용, thread-local result merge 비용, IPC 비용이 모두 포함되기 때문이다. 따라서 hybrid 가 모든 조건에서 가장 빠른 구조라고 주장하는 것은 위험하다. 대신 충분히 큰 workload 에서 process 와 thread 의 역할 분리를 통해 병렬화 효과를 얻을 수 있으며, OS 개념 설명력이 높은 확장 구조로 해석하는 것이 적절하다.

표 24. 4-Worker 주요 실행 구조 성능 비교

Case	Workers	Avg Time	Speedup	Efficiency	해석
seq	1	12.111s	1.000x	100.0%	기준 실행시간
thread_4_reduce	4	3.236s	3.743x	93.6%	thread-local reduce 기반 비교군
process_4_shm	4	3.222s	3.759x	94.0%	큰 독립 작업과 shm 결과 전달로 높은 효율
hybrid_2x2_shm	4	3.222s	3.758x	94.0%	process 내부 pthread 를 결합한 계층적 병렬화
pipeline_4_final	4	3.275s	3.698x	92.5%	queue 기반 구조 비교군

위 결과는 Docker Desktop Ubuntu 22.04 Linux VM 에서 실제 Monte Carlo workload 를 대상으로 trials=120,000,000, steps=50, 각 case 5 회 반복 조건으로 측정한 결과이다. 따라서 수치는 native Linux 의 절대 성능으로 일반화하기보다, 동일한 Docker Desktop Linux VM 환경에서 실행 구조별 상대 차이를 비교한 결과로 해석한다.

최신 실제 Monte Carlo strong scaling 결과에서 hybrid_2x2_shm 은 4-worker 조건에서 평균 3.222s, speedup 3.758x, efficiency 94.0%를 기록하였다. 이는 child process 내부에서 pthread worker 를 사용하는 계층적 병렬 구조가 충분히 큰 workload 에서는 효과적으로 동작할 수 있음을 보여준다.

그러나 hybrid_2x2_shm, process_4_shm, thread_4_reduce 의 평균 시간 차이는 크지 않다. 따라서 hybrid 가 항상 최적이라고 해석하기보다는, process isolation 과 thread 병렬성을 결합한 구조적 trade-off 를 보여주는 실행 mode 로 설명하는 것이 적절하다.

5-7. Pipeline 결과 분석

Pipeline 결과 분석에서는 queue 기반 작업 분배와 병합 방식에 따라 성능과 overhead 가 어떻게 달라지는지 확인하였다. 최종 성능 분석은 Docker Desktop 기반 Ubuntu 22.04 Linux VM 에서 실제 Monte Carlo workload 를 대상으로 수행한 반복 측정 결과를 기준으로 하였다. 실험 조건은 trials=120,000,000, steps=50, profile=default, pre_work=0, post_work=0, 각 case 5 회 반복이며,

CPU affinity 를 활성화한 상태에서 측정하였다. 이 결과는 native Linux 의 절대 성능으로 일반화하기보다, 동일한 Docker Desktop Linux VM 환경 안에서 Pipeline mode 의 확장성과 queue overhead 를 비교하기 위한 값으로 해석하였다.

표 25. Pipeline strong scaling 결과

Case	Workers	Avg Time	Stdev	Speedup	Efficiency	CPU%	Util/Worker
pipeline_4_final	4	3.275s	0.137%	3.698%	92.5%	399.0%	99.8%
pipeline_8_final	8	2.177s	0.058s	5.564%	69.6%	797.6%	99.7%

pipeline_4_final 은 평균 실행 시간 3.275s, speedup 3.698x, efficiency 92.5%로 측정되었다. worker 당 CPU utilization 도 99.8%로 나타나, 4-worker 조건에서 pipeline 구조가 실제 Monte Carlo workload 를 충분히 병렬로 처리하고 있음을 확인할 수 있다. 다만 같은 4-worker 주요 구조와 비교하면 pipeline 은 중앙 queue 를 사용하므로 queue scheduling 과 worker 관리 비용이 추가된다. 따라서 pipeline 결과는 단순히 가장 빠른 구조를 찾기 위한 것이 아니라, queue 기반 작업 분배가 성능과 overhead 에 어떤 영향을 주는지 확인하기 위한 결과로 해석해야 한다.

pipeline_8_final은 평균 실행 시간 2.177s, speedup 5.564x, efficiency 69.6%로 측정되었다. CPU%는 797.6%, worker 당 utilization 은 99.7%로 매우 높게 나타났지만, efficiency 는 4-worker 조건보다 크게 감소하였다. 이는 CPU utilization 이 높다는 사실이 곧 이상적인 scaling 을 의미하지는 않는다는 점을 보여준다. 모든 worker 가 CPU 를 사용하고 있더라도, queue 관리, scheduling, worker 증가에 따른 overhead 가 커지면 실행시간은 $T1/N$ 에 비례해 줄어들지 않을 수 있다.

Final Reduce와 Interactive Merge 는 병합 방식의 목적이 다르다. Final Reduce 는 모든 worker 가 계산을 끝낸 뒤 마지막에 한 번에 병합하는 구조이므로 단순하고 동기화 비용이 작다. 반면 Interactive Merge 는 worker 가 생성한 partial result 를 MergeQueue 에 전달하고 aggregator thread 가 실행 중간에 병합하는 구조이므로, 실행 중 partial result 를 계속 집계할 수 있다. 그러나 이 과정에서 queue 접근, mutex lock/unlock, condition variable wait/signal, aggregator thread 실행 비용이 추가된다.

표 26. Final Reduce 와 Interactive Merge 결과 해석

방식	장점	비용	적합한 상황	결과 해석
Final Reduce	구조 단순, 동기화 비용 작음	실행 중 중간 결과 없음	최종 결과만 필요한 batch 작업	pipeline strong scaling 의 기본 기준으로 사용
Interactive Merge	실행 중 partial result 집계 가능	queue, lock, condition variable, aggregator 비용	지속 집계와 monitoring 이 필요한 작업	느리게 측정되어도 실패가 아니라 기능과 overhead 의 trade-off

결과적으로 Pipeline 실험은 Interactive Merge 가 Final Reduce 보다 항상 빠르다는 것을 보이기 위한 실험이 아니다. Pipeline mode 의 핵심은 batch 단위 작업 분배, producer-consumer 구조, queue

synchronization overhead 를 확인하는 데 있다. Final Reduce 는 최종 결과만 필요한 작업에서 단순하고 안정적인 병합 방식으로 사용할 수 있고, Interactive Merge 는 실행 중 중간 결과 집계와 집계가 필요한 상황에서 의미가 있다. 다만 Interactive Merge 는 queue, condition variable, aggregator thread 로 인한 overhead 가 추가되므로 workload 크기, batch size, worker 수에 따라 Final Reduce 보다 느릴 수 있다. 따라서 본 프로젝트에서는 Pipeline 결과를 단순 성능 순위가 아니라, producer-consumer 구조에서 queue scheduling 과 merge overhead 가 성능에 미치는 영향을 보여주는 trade-off 로 해석하였다.

6. 문제 발생 및 해결 과정

6-1. 문제-원인-해결 요약

본 프로젝트는 처음부터 최종 구조가 완성된 상태로 진행된 것이 아니라, 구현과 실험을 반복하면서 확인된 문제를 해결하는 방식으로 발전하였다. 초기에는 Monte Carlo trial 을 여러 worker 로 나누어 계산하는 단순 병렬 구조에서 출발하였다. 그러나 실제 구현 과정에서 thread race condition, mutex lock contention, process 간 결과 공유 문제, IPC overhead, pipeline queue overhead, worker 수 증가에 따른 efficiency 감소 문제가 확인되었다.

따라서 최종 시스템은 단순 병렬 계산 프로그램이 아니라, 운영체제의 주요 실행 구조인 thread, process, IPC, hybrid, pipeline 을 동일한 CPU-bound workload 위에서 비교할 수 있는 실험 시스템으로 확장되었다. 본 장에서는 프로젝트 진행 중 확인한 문제를 원인과 해결 방향 중심으로 정리한다.

표 27. 문제 발생 및 해결 과정 요약

문제	원인	해결 방향	최종 반영 구조
이미지 필터 병렬 처리의 성능 원인 분리 어려움	파일 I/O, 메모리 접근, cache 영향이 큼	CPU-bound 반복 계산 workload 로 변경	Monte Carlo 차량 추종 위험 계산
sequential 실행만으로는 병렬화 분석 불가	단일 실행 흐름만 존재	모든 병렬 mode 의 기준 실행 시간으로 사용	Sequential baseline
thread 만으로는 OS 개념 설명이 제한됨	process, IPC, address space 차이 분석 불가	child process 구조 추가	Process mode
여러 thread 의 공유 결과 갱신 문제	shared result 에 대한 race condition	nosync, mutex, reduce 비교	Synchronization modes
mutex 사용 시 성능 저하	trial 마다 lock/unlock 발생	thread-local result 후 병합	Reduce 방식
process 결과 공유 문제	parent 와 child 의 주소 공간 분리	IPC 방식 추가	pipe, shared memory
process 와 thread 결합 구조 부족	각각의 장단점만 따로 분석됨	계층적 병렬 구조 추가	Hybrid mode
final reduce 만으로 pipeline 구조 설명 부족	실행 중 partial result 병합 구조 없음	queue 기반 producer-consumer 구조 추가	Pipeline mode
CPU 활용률 근거 부족	실행 시간만으로 core 활용 판단 어려움	CPU%, Util/Worker, core 별 utilization 측정	/usr/bin/time -v, mpstat

8-worker efficiency 감소	worker 증가에 따른 scheduling, queue, merge overhead 증가	speedup 과 CPU utilization 을 분리해 분석	Strong scaling 분석
------------------------	--	------------------------------------	-------------------

6-2. 초기 Workload 선정 문제와 해결

초기 주제는 이미지 필터 병렬 처리였다. 이미지 필터는 이미지를 2 차원 배열로 보고 영역을 나누어 처리할 수 있으므로 thread 나 process 기반 병렬화가 가능하다. 그러나 운영체제 병렬 처리 구조를 분석하기 위한 workload 로는 한계가 있었다.

가장 큰 문제는 실행 시간에 여러 요인이 섞인다는 점이다. 이미지 처리에서는 파일을 읽고 쓰는 I/O 비용, 이미지 배열에 접근하는 메모리 비용, cache locality, memory bandwidth 가 전체 성능에 영향을 준다. 이 경우 worker 수를 늘렸을 때 실행 시간이 감소하더라도, 그 원인이 CPU 병렬화 때문인지 메모리 접근 패턴 때문인지 명확하게 분리하기 어렵다.

이를 해결하기 위해 최종 workload 를 Monte Carlo 기반 차량 추종 위험 계산으로 변경하였다. 이 workload 는 각 trial 이 독립적으로 실행되고 반복 계산 중심으로 구성된다. 또한 trial range 를 thread, process, hybrid, pipeline 구조로 나누기 쉽고, 결과는 hist_sum, checksum, valid 로 검증할 수 있다.

따라서 Monte Carlo 차량 추종 위험 계산은 실제 차량 시뮬레이터를 정밀하게 구현하기 위한 모델이 아니라, 운영체제 실행 구조별 병렬 처리 효과를 비교하기 위한 CPU-bound 실험 workload 로 사용된다.

6-3. Thread 병렬화 문제와 해결

Thread mode 에서는 여러 pthread worker 가 하나의 process 내부에서 실행된다. 이 구조는 생성 비용이 작고 같은 주소 공간을 공유하므로 결과 접근이 빠르다는 장점이 있다. 그러나 여러 thread 가 같은 Result 구조체를 동시에 갱신하면 race condition 이 발생할 수 있다.

예를 들어 여러 thread 가 total_trials, collision_count, histogram, checksum 을 동시에 갱신하면 read-modify-write 과정이 겹칠 수 있다. 이 경우 일부 update 가 손실되어 hist_sum 이 전체 trials 와 일치하지 않거나 checksum mismatch 가 발생한다. 따라서 thread mode 에서는 실행 시간이 짧은지만 확인해서는 안 되고, 반드시 valid, hist_sum, checksum 을 함께 확인해야 한다.

이를 검증하기 위해 nosync, mutex, reduce 세 가지 방식을 비교하였다. nosync 는 동기화 없이 shared result 를 갱신하는 방식으로, race condition 을 보여주는 실패 사례이다. mutex 는 critical section 을 lock 으로 보호하여 정확성을 보장하지만, trial 마다 lock/unlock 이 반복되면 lock contention 이 발생한다. reduce 는 각 thread 가 thread-local result 를 계산하고, 모든 thread 가 종료된 뒤 main thread 가 한 번에 병합하는 방식이다.

결과적으로 thread mode 에서 중요한 개선 방향은 단순히 lock 을 추가하는 것이 아니라, hot loop 에서 shared write 를 제거하는 것이다. Reduce 방식은 공유 데이터 접근을 줄여 정확성을 유지하면서 synchronization overhead 를 줄이는 구조로 사용되었다.

6-4. Process 와 IPC 구조 추가 이유

Thread 구조만으로는 운영체제 프로젝트에서 요구하는 child process, 독립 주소 공간, IPC 구조를 충분히 설명하기 어렵다. Thread 는 같은 process 내부에서 실행되므로 메모리를 자연스럽게 공유하지만, process 는 독립된 address space 를 사용한다. 따라서 child process 가 계산한 결과는 parent process 의 메모리에 자동으로 반영되지 않는다.

이 문제를 해결하기 위해 process mode 를 추가하였다. Process mode 에서는 parent process 가 fork()로 child process 를 생성하고, 각 child 가 독립된 trial range 를 계산한다. 계산이 끝난 child 는 결과를 parent 에게 전달해야 하므로 pipe IPC 또는 shared memory IPC 가 필요하다.

Pipe IPC 는 child 가 계산한 Result 를 pipe 에 write()하고 parent 가 이를 read()하여 병합하는 구조이다. 이 방식은 메시지 전달 흐름이 명확하지만, kernel buffer 를 통한 read/write overhead 가 발생한다. Shared memory IPC 는 anonymous mmap() 기반 shared result table 을 사용하고, 각 child 가 자기 slot 에 결과를 기록하는 방식이다. Parent 는 waitpid() 이후 각 slot 을 읽어 최종 결과에 병합한다.

따라서 process mode 의 핵심은 단순히 실행 시간을 줄이는 것이 아니라, child process 의 독립 주소 공간과 IPC 비용을 실험적으로 확인하는 데 있다.

6-5. Hybrid 구조 추가 이유

Process mode 와 thread mode 를 각각 구현한 뒤에도 단일 구조 비교의 한계가 남았다. Thread 는 lightweight 병렬화와 메모리 공유가 장점이지만 race condition 문제가 있다. Process 는 독립성과 안정성이 장점이지만 IPC overhead 가 있다. 따라서 두 구조를 따로 비교하는 것만으로는 process 와 thread 를 함께 사용하는 계층적 병렬 구조를 설명하기 어렵다.

이를 해결하기 위해 hybrid mode 를 추가하였다. Hybrid mode 에서는 parent process 가 child process 를 생성하고, 각 child process 내부에서 다시 pthread worker 를 생성한다. Process 는 큰 작업 그룹을 나누고, thread 는 child 내부에서 세부 trial range 를 병렬로 처리한다. 이후 child 내부에서 thread-local result 를 child-local result 로 병합하고, parent 에게 IPC 를 통해 전달한다.

Hybrid mode 는 process isolation 과 thread parallelism 을 함께 보여줄 수 있다는 장점이 있다. 그러나 child process 생성, pthread 생성 및 join, child-local merge, IPC 비용이 모두 포함되므로 구조가 복잡하고 overhead 도 증가할 수 있다. 따라서 hybrid 는 항상 가장 빠른 구조가 아니라, process 와 thread 의 역할 분리를 보여주는 계층적 병렬 구조로 해석하는 것이 적절하다.

6-6. Pipeline 구조 추가 이유

기본 thread reduce 방식은 단순하고 효율적이다. 그러나 모든 worker 가 계산을 끝낸 뒤 마지막에 한 번 병합하는 구조이므로, 실제 시스템에서 자주 등장하는 producer-consumer 구조를 설명하기에는 부족하다. 또한 작업이 batch 단위로 분배되거나 실행 중 partial result 를 집계해야 하는 경우에는 queue 기반 구조가 필요하다.

이를 위해 pipeline mode 를 추가하였다. Pipeline mode 에서는 preprocessor 가 trial 을 batch 단위로 생성하고 TaskQueue 에 넣는다. Worker thread 는 TaskQueue 에서 batch 를 꺼내 Monte Carlo 계산을 수행하고, 결과를 final reduce 방식으로 병합하거나 MergeQueue 를 통해 aggregator 에게 전달한다.

TaskQueue 와 MergeQueue 는 여러 thread 가 함께 접근하는 공유 queue 이다. 따라서 queue 내부의 buffer, head, tail, count, closed 와 같은 상태는 mutex 로 보호해야 한다. 또한 queue 가 비어 있거나 가득 찬 경우 busy waiting 을 피하기 위해 condition variable 을 사용한다.

Pipeline mode 의 목적은 항상 가장 빠른 구조를 만드는 것이 아니다. Final Reduce 는 구조가 단순하고 동기화 비용이 작지만 실행 중 partial result 를 확인하기 어렵다. Interactive Merge 는 실행 중 partial result 집계 가능하지만 MergeQueue, condition variable, aggregator thread overhead 가 추가된다. 따라서 pipeline 결과는 producer-consumer 구조에서 queue scheduling 과 merge overhead 가 성능에 어떤 영향을 주는지 보여주는 실험으로 해석한다.

6-7. CPU Utilization 측정 추가 이유

초기 성능 비교는 주로 실행 시간과 speedup 중심으로 진행되었다. 그러나 실행 시간만으로는 실제 CPU core 가 얼마나 사용되었는지 확인하기 어렵다. 예를 들어 실행 시간이 줄어들지 않았다고 해서 CPU 가 사용되지 않은 것은 아니며, 반대로 CPU utilization 이 높다고 해서 실행 시간이 이상적으로 줄어드는 것도 아니다.

이를 확인하기 위해 최종 실험에서는 CPU%, Util/Worker, core 별 utilization 을 함께 측정하였다. CPU%는 전체 worker 가 사용한 CPU 총량을 나타내고, Util/Worker 는 이를 worker 수로 나눈 값이다. core 별 utilization 은 실제로 각 core 가 균일하게 사용되었는지를 확인하기 위해 사용한다.

최종 결과에서 4-worker 주요 구조는 worker 당 CPU utilization 이 약 99.2~99.8% 수준으로 측정되었다. 이는 4-worker 조건에서 각 worker 가 CPU-bound workload 를 충분히 수행하고 있음을 의미한다. 반면 8-worker 조건에서는 CPU utilization 은 높게 유지되었지만 efficiency 는 감소하였다. 이 결과는 CPU 를 바쁘게 사용하는 것과 실행 시간이 이상적으로 감소하는 것이 서로 다른 문제임을 보여준다.

6-8. 문제 해결 과정 요약

본 프로젝트의 개선 과정은 단순히 기능을 추가한 과정이 아니라, 운영체제 병렬 처리에서 발생하는 문제를 확인하고 이를 설명할 수 있는 구조를 추가한 과정이다. Thread mode 에서는 race condition 과 synchronization overhead 를 분석하였다. Process mode 에서는 child process 와 IPC 구조를 분석하였다. Hybrid mode 에서는 process 와 thread 의 계층적 결합을 분석하였다. Pipeline mode 에서는 producer-consumer 구조와 queue synchronization overhead 를 분석하였다.

최종적으로 본 프로젝트는 동일한 CPU-bound workload 를 여러 운영체제 실행 구조로 처리하고, 정확성, 실행 시간, speedup, efficiency, CPU utilization, overhead 를 함께 비교하는 실험 시스템으로 정리된다. 핵심 결론은 특정 구조가 항상 가장 빠르다는 것이 아니라, workload 특성, worker 수, synchronization 방식, IPC 방식, queue 구조에 따라 성능과 overhead 가 달라진다는 점이다.

7. 역할분담 및 수행 과정

7-1. 역할분담 개요

본 프로젝트는 구현 영역과 분석 영역을 기준으로 역할을 분담하였다. 각 팀원은 하나의 기능만 담당한 것이 아니라, 자신이 맡은 운영체제 구조에 대해 구현 내용 정리, 실험 결과 해석, 보고서 작성, 발표자료 구성까지 함께 수행하였다.

역할분담은 코드 작성량만을 기준으로 하지 않았다. Monte Carlo workload 설계, thread synchronization 분석, process/IPC 구조 정리, pipeline queue 분석, 실험 결과 검증, 보고서 통합, 발표 준비를 모두 프로젝트 기여 요소로 보았다. 특히 최종 보고서에서는 각자의 구현 파트가 단순히 나열되지 않도록, 전체 흐름을 문제 → 원인 → 해결 → 검증 구조로 연결하였다.

표 28. 팀원별 역할분담

팀원	담당 영역	주요 수행 내용	보고서 담당	기여도
정재훈	총괄 / 문서 통합 / 구조 개선 흐름	프로젝트 전체 구조 정리, 보고서 목차 구성, 문제-원인-해결 과정 작성, 발표 흐름 설계, 결론 및 AI 사용 기록 작성	초록, 시스템 개요, 문제 발생 및 해결 과정, 역할분담, AI 사용 기록, 결론	25%
유지원	Simulation / 성능 실험	Monte Carlo trial 구조 정리, deterministic checksum 설계, workload 조건 정리, strong scaling 및 CPU utilization 결과 분석	Trial 계산 구조, Test Vector, 실험 환경, 성능 결과 분석	20%
성도연	Thread Synchronization / Pipeline	nosync, mutex, reduce 비교, race condition 분석, TaskQueue, MergeQueue, condition variable 구조 정리	Thread/Synchronization, Pipeline, Synchronization 결과 분석	20%
김태환	Process / IPC / Hybrid	fork, waitpid, pipe IPC, shared memory IPC, hybrid 구조 정리	Process, IPC, Hybrid 결과 분석	20%
정종근	실행 검증 / 결과 정리	실행 명령 확인, 결과 파일 경로 검수, 표/그림 번호 검수, CPU utilization 표 검수, 발표 결과 파트 보조	실험 환경 보조, CPU utilization 표 검수, 부록 정리	15%

7-2. 정재훈 담당 수행 내용

정재훈은 프로젝트의 전체 구조 기획과 보고서 통합을 담당하였다. 초기 주제에서 최종 Monte Carlo 기반 CPU-bound workload 로 변경된 배경을 정리하고, 프로젝트가 단순 병렬 처리 구현에서 운영체제 실행 구조별 trade-off 분석으로 발전한 과정을 문서화하였다.

또한 최종 보고서 목차를 구성하고, 각 팀원이 담당해야 할 내용을 장별로 분배하였다. 특히 문제 발생 및 해결 과정에서는 thread race condition, mutex lock contention, process 간 결과 공유 문제, IPC 필요성,

pipeline queue overhead, CPU utilization 측정 필요성을 하나의 흐름으로 연결하였다. 이를 통해 각 기능이 단순히 추가된 것이 아니라, 실제 문제를 해결하기 위해 확장되었다는 점을 설명하였다.

발표자료 구성에서도 전체 흐름을 담당하였다. 발표는 5 명이 모두 한 번씩 참여하도록 구성하고, 프로젝트 목표, 시스템 구성, 문제 개선 과정, 실험 결과, 최종 결론이 자연스럽게 이어지도록 슬라이드 순서를 설계하였다.

7-3. 유지원 담당 수행 내용

본 프로젝트의 중간 발표를 전담하여 초기 시스템 설계 배경과 Monte Carlo 병렬화 모델의 타당성을 전달하였다. 이와 더불어, Trial 단위의 deterministic seed 계산 구조와 checksum 검증 체계를 이론적으로 정립하였고, default, uniform, skewed 등 분석 목적에 따른 workload 조건 및 평가 지표를 표와 글로 명시하였다. 수행일지를 담당하여 작성하였다.

이와 함께 Linux 환경에서 1 억 2 천만(120,000,000) 개의 trial 과 50 step 조건을 기반으로 1, 2, 4, 8 worker 에 대한 실제 Monte Carlo strong scaling 및 core 별 CPU utilization 실험을 직접 실행하여 데이터를 측정하였다. 반복 실험을 통해 추출된 수치들을 보고서의 최종 성능 데이터와 꼼꼼하게 대조·검수하였으며, 이를 바탕으로 실행 구조별 속도 향상(Speedup), 효율성(Efficiency), Worker 당 CPU 활용률(CPU%) 그래프를 직접 시각화하였다. 최종적으로 4-worker 환경의 높은 자원 포화도 (92.5 ~ 94.0% efficiency)와 8-worker 환경의 효율 감소 데이터를 비교 분석하여 코어 활용률과 실제 실행 시간 단축은 서로 다른 지표임을 수치 레벨에서 입증하는 역할을 수행하였다.

7-4. 성도연 담당 수행 내용

Thread Synchronization 과 Pipeline Queue 파트를 맡아, 병렬 처리 과정에서 공유 Result 구조체 접근 문제가 어떻게 발생하고 이를 어떤 방식으로 해결할 수 있는지 정리하였다. thread mode 에서는 nosync, mutex, reduce 방식을 비교하여 race condition 발생 원인, mutex 사용 시 lock contention 문제, thread-local result 를 이용한 reduce 구조를 분석하였다. 또한 nosync 는 성능 좋은 방식이 아니라 valid=0 과 checksum 불일치를 통해 synchronization 필요성을 보여주는 실패 사례로 해석하였다.

Pipeline 파트에서는 TaskQueue 와 MergeQueue 를 중심으로 producer-consumer 구조를 설명하고, mutex 와 condition variable 이 queue 상태 보호와 empty/full 대기 처리에 사용되는 과정을 정리하였다. 마지막으로 Final Reduce 와 Interactive Merge 의 차이를 비교하여, Interactive Merge 가 느리게 측정되더라도 실패가 아니라 중간 결과 집계 기능을 위한 queue, condition variable, aggregator thread overhead 로 해석해야 함을 분석하였다.

7-5. 김태환 담당 수행 내용

Process / IPC / Hybrid 구조를 담당하였다. 주요 수행 내용은 process mode 의 필요성 정리, fork()와 waitpid() 기반 child process 실행 흐름 설명, pipe IPC 와 shared memory IPC 비교, hybrid mode 의 process-thread 결합 구조 정리이다.

또한 process mode 와 hybrid mode 의 실행 결과를 바탕으로 child process 가 계산한 결과가 IPC 를 통해 parent process 로 전달되고, 최종적으로 merge 와 checksum validation 이 수행되는 구조를 정리하였다. 이를 통해 thread, process, hybrid 구조의 차이와 각 방식의 장단점 및 overhead 를 비교 분석하였다.

7-6. 정종근 담당 수행 내용

최종 실험 결과 정리와 검증 결과 요약에 담당하였다. Strong scaling 결과, CPU utilization 결과, core 별 활용률 결과를 보고서와 발표자료에 반영할 수 있도록 정리하였으며, 1, 2, 4, 8 worker 조건에서 실행 시간이 어떻게 감소하는지와 4-worker 조건에서 주요 구조들이 어느 정도의 efficiency 를 보이는지 확인하였다.

특히 4-worker 조건에서 주요 구조들이 92.5~94.0% 수준의 efficiency 를 보이고, worker 당 CPU utilization 이 99.2~99.8% 수준으로 측정된 결과를 정리하였다. 또한 8-worker 조건에서는 CPU utilization 이 높게 유지되더라도 efficiency 가 감소한다는 점을 함께 정리하여, CPU 를 많이 사용하는 것과 실행 시간이 이상적으로 감소하는 것은 같은 의미가 아님을 결과 해석에 반영하였다.

문서화 과정에서는 speedup, efficiency, CPU%, Util/Worker, core utilization 과 같은 측정 지표가 보고서와 발표자료에서 일관되게 사용되도록 정리하였다. 최종 발표에서는 strong scaling 과 CPU utilization 결과를 요약하는 파트를 담당하여, 프로젝트의 최종 검증 결과가 단순 수치 나열이 아니라 병렬화 효율과 CPU 활용률의 차이를 설명하는 근거로 전달될 수 있도록 구성하였다.

8. AI 사용 기록

8-1. AI 사용 목적

본 프로젝트에서는 ChatGPT 와 Codex 를 보조 도구로 사용하였다. AI 는 프로젝트 결과를 대신 생성하는 용도가 아니라, 코드 구조 분석, 구현 방향 검토, 보고서 첨삭, 발표자료 구성, 역할분담 정리 등을 보조하는 도구로 활용하였다.

프로젝트가 진행되면서 실행 구조가 Sequential, Thread, Process, Hybrid, Pipeline 으로 확장되었고, 각 구조는 서로 다른 운영체제 개념과 overhead 를 포함하게 되었다. 이 과정에서 코드 구조와 실험 결과를 단순히 나열하면 보고서의 논리 흐름이 약해질 수 있었다. 따라서 AI 를 사용하여 각 구조가 왜 추가되었는지, 어떤 문제를 해결하는지, 실험 결과를 어떻게 해석해야 하는지 정리하였다.

또한 코드 분석에도 AI 를 활용하였다. Thread synchronization, process/IPC, hybrid, pipeline queue 구조가 각각 어떤 역할을 하는지 파악하고, 각 파일의 기능과 실행 흐름을 분석하는 데 사용하였다. 특히 race condition, mutex lock contention, shared memory IPC, queue synchronization, CPU utilization 측정 방식이 코드에서 어떻게 구현되는지 검토하는 과정에서 AI 를 보조적으로 활용하였다.

보고서 작성 과정에서는 문장 첨삭과 구조 개선에도 AI 를 사용하였다. 초록, 시스템 개요, 문제 발생 및 해결 과정, 결론 등에서 문장이 장황해지지 않도록 정리하고, 공학적 글쓰기 방식에 맞게 문제 → 원인 → 해결 → 검증 흐름으로 수정하는 데 활용하였다. 다만 AI 가 제안한 문장은 그대로 제출하지 않고, 실제 구현 내용과 실험 결과에 맞게 팀원이 검토한 뒤 반영하였다.

8-2. 주요 AI 사용 내역

표 29. AI 사용 내역

사용 도구	사용 목적	활용 범위	검증 방식
ChatGPT	보고서 목차 재구성	초록, 시스템 개요, 문제 발생 및 해결 과정, 결론 구조 정리	팀원 검토 후 문서에 반영
ChatGPT	보고서 문장 첨삭	문장 압축, 공학적 문체 정리, 논리 흐름 개선	실제 구현 내용과 비교 후 수정
ChatGPT	코드 구조 분석 보조	thread, process, IPC, hybrid, pipeline 구조의 역할 분석	코드 파일과 실행 결과 기준으로 확인
ChatGPT	실험 결과 해석 보조	speedup, efficiency, CPU utilization, overhead 해석	최종 CSV 와 README 결과 기준으로 대조
ChatGPT	발표자료 구성	10 분 발표 기준 슬라이드 목차와 발표자별 분담 구성	발표 흐름 리허설로 검토
Codex	코드 수정 방향 검토	thread/process/hybrid/pipeline 성능 비교를 위한 구현 보완 방향 확인	make, make test, 실행 결과로 검증

Codex	코드 분석 및 수정 보조	실험 스크립트, CSV 출력, CPU utilization 측정 흐름, test script 검토	valid, hist_sum, checksum, CSV 결과 확인
Canva	발표자료 시각화	시스템 흐름도, 평가 지표, 결과 그래프 배치	팀원 발표자료 검토
Google Docs	보고서 공동 작성	팀원별 작성, 문체 통일, 표/그림 정리	최종 교차 검수

8-3. 주요 Prompt 사용 방향

표 30. 주요 Prompt 사용 방향

목적	Prompt 방향	결과 활용
코드 구조 분석	현재 프로젝트 코드에서 thread, process, IPC, hybrid, pipeline 구조가 어떻게 연결되는지 분석 요청	프로그램 구성 및 문제 해결 과정 작성에 반영
synchronization 분석	nosync, mutex, reduce 방식의 차이와 race condition 발생 원인 분석 요청	Thread synchronization 설명에 반영
IPC 구조 분석	pipe IPC와 shared memory IPC의 차이, parent-child 결과 전달 흐름 분석 요청	Process/IPC 설명에 반영
pipeline 분석	TaskQueue, MergeQueue, condition variable, final/interactive merge 구조 분석 요청	Pipeline 구조와 overhead 설명에 반영
보고서 구조 정리	최신 README와 실험 결과를 기준으로 보고서 목차를 재구성하도록 요청	최종 보고서 목차와 역할분담 정리
보고서 첨삭	공학적 글쓰기 기준으로 장황한 문장을 줄이고 논리 흐름을 개선하도록 요청	초록, 시스템 개요, 6 장, 결론 문장 수정
역할분담 구성	5 명이 모두 보고서 작성과 발표에 참여하도록 파트별 역할분담 요청	7 장 역할분담 및 발표자료 분담에 반영
Codex 용 코드 수정 방향	CPU utilization, process/thread/hybrid/pipeline 비교가 가능하도록 구현 보완 방향 요청	코드 수정 및 실험 script 개선 참고
발표자료 구성	10 분 발표, 5 명 전원 1 회 발표 조건으로 PPT 목차와 시간 배분 요청	Canva 발표자료 구성에 반영
결과 해석 보조	CPU utilization 과 speedup/efficiency 의 차이를 설명하도록 요청	실험 결과 분석과 결론에 반영

9. 결론

본 프로젝트는 Monte Carlo 기반 차량 추종 위험 계산을 CPU-bound workload 로 설정하고, 이를 여러 운영체제 실행 구조로 처리하면서 정확성, 성능, CPU 활용률, overhead 를 비교한 실험 프로젝트이다. 본 프로젝트에서 차량 추종 위험 계산은 실제 차량 시뮬레이터 구현 자체가 목적이 아니라, child process, POSIX thread, synchronization, IPC, hybrid, pipeline 구조가 병렬 처리 성능에 어떤 영향을 주는지 분석하기 위한 실험용 workload 로 사용되었다.

최종 시스템은 동일한 Monte Carlo workload 를 Sequential, Thread, Process, Hybrid, Pipeline 구조로 실행하도록 구성하였다. Sequential mode 는 모든 성능 비교의 기준 실행 시간으로 사용되었고, Thread mode 는 pthread 기반 병렬 처리와 synchronization 문제를 분석하는 데 사용되었다. Process mode 는 child process 와 parent process 의 독립 주소 공간, 그리고 결과 전달을 위한 IPC 구조를 확인하는 데 사용되었다. Hybrid mode 는 process 와 thread 를 계층적으로 결합한 구조이며, Pipeline mode 는 TaskQueue 와 MergeQueue 를 기반으로 producer-consumer 구조와 queue synchronization overhead 를 분석하기 위해 사용되었다.

구현 과정에서 가장 먼저 확인한 문제는 thread 기반 병렬화의 정확성 문제이다. nosync 방식은 shared result 를 보호하지 않기 때문에 race condition 이 발생할 수 있고, 이 경우 실행 시간이 짧아 보여도 hist_sum, checksum, valid 기준을 만족하지 못한다. 따라서 본 프로젝트에서는 정확성이 보장되지 않은 결과를 성능 비교 대상으로 사용하지 않았다. mutex 방식은 critical section 을 보호하여 정확성을 회복할 수 있지만, 반복 계산 과정에서 lock contention 이 발생할 수 있다. 이를 개선하기 위해 worker 별 local result 를 계산한 뒤 마지막에 병합하는 reduce 구조를 사용하였다. Reduce 방식은 hot loop 에서 shared write 를 줄여 정확성과 성능을 함께 확보하는 방식으로 정리된다.

Process 구조에서는 thread 와 다른 문제가 발생하였다. Thread 는 같은 address space 를 공유하지만, process 는 독립된 address space 를 사용한다. 따라서 child process 가 계산한 결과는 parent process 에 자동으로 반영되지 않으며, pipe 또는 shared memory 와 같은 IPC 가 필요하다. Pipe IPC 는 child 가 결과를 write 하고 parent 가 read 하는 명시적인 전달 구조를 제공한다. Shared memory IPC 는 child 별 result slot 을 사용하여 결과 전달 비용을 줄일 수 있다. 이를 통해 process mode 는 단순한 속도 비교 대상이 아니라, process isolation 과 IPC overhead 를 분석하기 위한 구조로 사용되었다.

Hybrid 구조는 process 와 thread 의 장점을 계층적으로 결합하기 위해 추가되었다. Parent process 가 child process 를 생성하고, 각 child 내부에서 pthread worker 가 세부 Trial range 를 병렬처리한다. 이 구조는 process 단위의 독립성과 thread 단위의 경량 병렬성을 동시에 보여줄 수 있다. 다만 child 생성, thread 생성, child-local merge, IPC 비용이 모두 포함되므로 항상 가장 빠른 구조라고 볼 수는 없다. Hybrid 는 특정 조건에서 좋은 성능을 낼 수 있지만, 구조적 복잡도와 overhead 를 함께 고려해야 하는 방식이다.

Pipeline 구조는 producer-consumer 모델을 실험하기 위해 추가되었다. Final Reduce 는 모든 worker 가 종료된 후 결과를 병합하므로 구조가 단순하고 overhead 가 작다. 반면 Interactive Merge 는 실행 중 partial result 를 MergeQueue 와 Aggregator 를 통해 병합할 수 있지만, queue 접근, mutex, condition variable, aggregator thread 에 따른 비용이 추가된다. 따라서 Interactive Merge 가 Final Reduce 보다 느리게 나타나더라도 이는 실패가 아니라, 중간 결과 집계 기능을 얻기 위해 synchronization overhead 를 지불하는 구조적 trade-off 로 해석해야 한다.

성능 측면에서 최종 실험은 실제 Monte Carlo workload 를 기준으로 진행되었다. 4-worker 주요 구조는 약 92.5~94.0% 수준의 efficiency 를 보였고, worker 당 CPU utilization 은 약 99.2~99.8% 수준으로 측정되었다. 이는 4 개 worker 조건에서 CPU-bound workload 가 각 worker 를 충분히 활용했고, 실행 시간 역시 N-core scaling 에 상당히 근접했음을 의미한다. 반면 8-worker 조건에서는 CPU utilization 은 높게 유지되었지만 efficiency 는 감소하였다. 이 결과는 CPU 를 바쁘게 사용하는 것과 실행 시간이 이상적으로 감소하는 것이 동일하지 않다는 점을 보여준다.

결론적으로 본 프로젝트는 특정 실행 구조가 항상 최적이라는 결론을 내리지 않는다. Thread, process, hybrid, pipeline 은 각각 다른 장점과 overhead 를 가진다. Thread 는 생성 비용이 낮고 shared address space 를 활용할 수 있지만 synchronization 문제가 있다. Process 는 독립 주소 공간을 제공하지만 IPC 비용이 필요하다. Hybrid 는 process 와 thread 를 결합할 수 있지만 구조가 복잡하다. Pipeline 은 동적 작업 분배와 중간 결과 집계에 유리하지만 queue synchronization overhead 가 발생한다.

따라서 병렬 처리 구조의 성능은 단순히 worker 수를 늘리는 것으로 결정되지 않는다. Workload 특성, worker 수, synchronization 방식, IPC 방식, queue 구조, 실행 환경에 따라 speedup 과 efficiency 는 달라진다. 본 프로젝트의 의의는 같은 CPU-bound workload 를 여러 운영체제 실행 구조로 구현하고, valid, hist_sum, checksum 을 통한 정확성 검증과 speedup, efficiency, CPU utilization, overhead 분석을 함께 수행하여 운영체제 병렬 처리 구조의 trade-off 를 공학적으로 확인했다는 점이다.

부록

A. 코드 구조

표 31. 주요 코드 파일과 역할

구분	파일	역할
Main entry	src/main.c	CLI 옵션을 파싱하고 실행 mode 를 선택한다.
Simulation	src/simulation.c	Monte Carlo Trial 계산, TTC 계산, risk bucket 반영을 수행한다.
Result	src/result.c	histogram, checksum, valid 등 결과 구조를 초기화하고 병합한다.
Metrics	src/metrics.c	실행 시간, stage time, CPU 관련 측정값을 기록한다.
Sequential	src/sequential.c	단일 실행 흐름의 baseline 을 제공한다.
Thread	src/thread_mode.c	pthread 기반 병렬 처리, nosync/mutex/reduce 구조를 수행한다.
Process	src/process_mode.c	fork(), waitpid() 기반 child process 병렬 처리를 수행한다.
Pipe IPC	src/ipc_pipe.c	child process 의 결과를 pipe 로 parent 에게 전달한다.
Shared Memory IPC	src/ipc_shm.c	shared memory result slot 을 통해 child 결과를 전달한다.
Hybrid	src/hybrid_mode.c	child process 내부에서 pthread worker 를 실행하는 hybrid 구조를 수행한다.
Pipeline	src/pipeline_mode.c	TaskQueue, Worker Pool, MergeQueue 기반 pipeline 실행을 수행한다.
TaskQueue	src/task_queue.c	batch 작업을 worker 에게 전달하는 producer-consumer queue 를 구현한다.
MergeQueue	src/merge_queue.c	partial result 를 aggregator 에게 전달하는 merge queue 를 구현한다.
Test script	scripts/test_modes.sh	주요 mode 의 정확성 검증을 자동화한다.

B. 실행 및 검증 명령어

make clean && make

make test

/usr/bin/time -v ./sim --mode thread --threads 4 --trials 120000000 --steps 50 --sync reduce

C. 결과 파일 경로

results/csv/real_montecarlo_scaling_2026_06_06_5repeat/real_scaling_summary.csv

results/csv/real_montecarlo_utilization_2026_06_06/real_utilization_summary.csv